EnlngDB

The Reference Book

A reference for the EnlngDB database engine, written in the words you type to it. Every script in this book was run with enlngdb 5.6.0, and the output shown is what it printed.

github.com/Aero99op/enlang-main · MIT License

Contents

1. What EnlngDB is 3

2. Scripts and statements 5

3. Tables and columns 9

4. Adding rows 15

5. Finding rows 20

6. Totals and groups 27

7. Joins 31

8. Calculations and functions 38

9. Subqueries and combining queries 46

10. Window functions 52

11. Changing and deleting rows 56

12. Types and rules for columns 60

13. Changing the shape: alter, views, indexes 67

14. Transactions and databases 72

15. More than SQL: files, preview, undo, speed 80

16. Triggers, procedures and helpful errors 87

17. The file on disk and the C interface 94

18. Errors and refusals 99

19. From SQL to EnlngDB 103

20. What EnlngDB does not have 108

CHAPTER 1

What EnlngDB is

EnlngDB is a small database engine that you talk to in plain words. You make a table, put rows in it, and ask for rows back with sentences like `find name from student where marks is more than 80`. The engine is one program, `enlngdb.exe`, written in C with no libraries beyond the C standard library.

What it is good for

- Learning how tables, queries and joins work, because the words say what they do.
- Small programs and scripts that need to keep a few tables in a file.
- Trying an idea quickly in the playground on the Enlangg site, which runs the same engine in your browser.

What it is not

EnlngDB opens a database file without reading its rows, and reads each row the first time a statement needs it; the rows it reads are in memory or, past a budget, in a file that the system brings into memory where it is used, so a database can be larger than memory (chapter 15). A script opens a database by itself; for many programs at once, `enlngdb --serve` keeps it open and lets users sign in, each with the rights they were granted (chapter 14). Chapter 20 lists everything it does not have, so that you do not have to find out by trying.

One script, one engine

A script is a text file that ends in `.enlngdb`. Each line is a statement. You run it like this:

TERMINAL

```
enlngdb shop.enlngdb
```

The first complete script of this book creates a table, puts three rows in it and asks for them back.

SCRIPT

```
create table student with id, name, marks
insert into student with id 1, name "Asha", marks 80
insert into student with id 2, name "Ravi", marks 90
```

```
insert into student with id 3, name "Meera", marks 70
find name, marks from student where marks is more than 75
```

OUTPUT

```
Query OK, 0 rows affected. Table 'student' created with 3 columns.

Query OK, 1 row affected

Query OK, 1 row affected

Query OK, 1 row affected

+------+-------+
| name | marks |
+------+-------+
| Asha | 80    |
| Ravi | 90    |
+------+-------+
2 row(s) in set
```

Every script in this book was run with the real engine, and the box under each script is what the engine printed. The only things left out are the time in brackets that the engine adds to most lines, such as `(0.04 ms)`, because it changes from run to run, and the line that says which database was opened. A script that is meant to fail is marked `SCRIPT WITH A MISTAKE`, and the engine's own message is shown under it. Scripts in one chapter share one database, so a table made in an early script is still there in the later ones. Each chapter starts with an empty database.

How the book is organised

Chapters 2 to 7 teach the everyday words: scripts, tables, rows, finding, totals and joins. Chapters 8 to 10 add calculations, subqueries and window functions. Chapters 11 to 14 cover changing data, rules for columns, views and transactions. Chapter 15 adds what SQL does not have: CSV files, a preview of a change, undo, and how the engine stays fast. Chapter 16 adds triggers, procedures and suggestions for mistyped words. Chapters 17 to 20 describe the file on disk, the C interface, what the engine refuses, and what is missing.

CHAPTER 2

Scripts and statements

Running a script

A script is a plain text file. By habit it ends in `.enlngdb`. You give it to the engine:

TERMINAL

```
enlngdb shop.enlngdb
enlngdb -e "show tables"
```

The first form runs every statement in the file. The second runs the statements that you type on the command line, with `;` between them. What `-e` changes is saved to the database it names: `enlngdb -e "use database shop; insert into pets values (3, 'Kit')"`. Without `use database` it has no database of its own, so a change is not kept, and it says so. A script can be written in any editor. One saved by Notepad (with its mark at the start of the file, or as "Unicode", which is UTF-16) is read the same as a plain UTF-8 file. A statement is one line. It can go on over several lines when each next line starts with a word that cannot start a statement: `from`, `where`, `and`, `or`, `order`, `group`, `having`, `limit`, `join` (and `left`, `inner` ...), `on`, `values`, `union`. An empty line always ends it:

TERMINAL

```
find name
  from pets
  where id is 1
     or id is 3
  order by name
::: `enlngdb --version` prints the version, and `enlngdb` alone prints a short help text.
## The header and the database name
A script may begin with the line `type enlngdb`. It says that what follows is a database script. In
     this book that line, and a line that chooses the database, are put in front of every script fo
    r you, so the scripts below show only the statements.
The line that chooses the database looks like this:
::: sh
use database shop
```

A database is a set of tables. When a script ends, the engine saves the database to a file called `<name>.edb` in the folder where you ran it, and the next script that says `use database shop` finds it again. A script without `use database` works in a database called `default`. If a statement in the script fails, the engine says so at the end and does not save anything from that run, so a half-finished script never leaves a half-finished file.

One statement per line

A statement ends at the end of the line. Two statements on one line are separated by a semicolon.

SCRIPT

```
create table items with id, title; insert into items with id 10, title "Keyboard"; find items
```

OUTPUT

```
Query OK, 0 rows affected. Table 'items' created with 2 columns.

Query OK, 1 row affected

+----+----------+
| id | title    |
+----+----------+
| 10 | Keyboard |
+----+----------+
1 row(s) in set
```

A statement can continue on the next line when you are inside brackets: the line must end with an opening bracket or a comma, or the next line must start with a closing bracket. Chapter 12 shows this with `create table`.

Comments

A comment starts with `#` or with `--` and runs to the end of the line. It can stand on its own line or follow a statement.

SCRIPT

```
-- how many items are there?
count records in items # the answer is a table with one cell
```

OUTPUT

```
+----------+
| count(*) |
+----------+
| 1        |
+----------+
1 row in set
```

Words the engine ignores

EnlngDB reads sentences, so a few small words may be left in or out without changing what a statement means. They are: `the`, `a`, `an`, `please`, `kindly`, `record`, `records`, `row`, `rows`, `entry`, `entries`, `data`, `item`, `items`. These three statements mean the same thing:

SCRIPT

```
find all records from items
find the records from items
please find items
```

OUTPUT

```
+----+----------+
| id | title    |
+----+----------+
| 10 | Keyboard |
+----+----------+
1 row(s) in set

+----+----------+
| id | title    |
+----+----------+
| 10 | Keyboard |
+----+----------+
1 row(s) in set

+----+----------+
| id | title    |
+----+----------+
| 10 | Keyboard |
+----+----------+
1 row(s) in set
```

A word is ignored only where a name or a keyword is not expected, and never inside quotes. A table that is really called `items` is still found by name in `from items`. If you are in doubt, leave the small words out.

Names, text, numbers

- A name is letters, digits and _. Table names and column names are written bare; a name with a space in it is not supported.
- `users.id` means the column `id` of the table `users`. You need this form only when two tables have a column of the same name.
- Text is written between double quotes or single quotes: `"Asha"` or `'Asha'`.
- A number is written plainly: `42`, `-100`, `3.14`.
- The words `true` and `false` are yes and no. The word `null` means that there is no value.

Showing a message

`display` prints text. It is useful in a script to mark where the output of one part begins.

SCRIPT

```
display "the second part starts here"
```

OUTPUT

```
the second part starts here
```

Naming things in a sentence

Most statements start with a verb: `create`, `insert`, `find`, `update`, `delete`, `alter`, `drop`, `show`, `describe`, `begin`. A few can also begin with a different word that means the same thing. For finding rows, `find`, `fetch`, `select` and `get` are the same word, and so are `show all records from` and `find all records from`.

SCRIPT

```
fetch title from items
select title from items
get from items
```

OUTPUT

```
+----------+
| title    |
+----------+
| Keyboard |
+----------+
1 row(s) in set

+----------+
| title    |
+----------+
| Keyboard |
+----------+
1 row(s) in set

+----+----------+
| id | title    |
+----+----------+
| 10 | Keyboard |
+----+----------+
1 row(s) in set
```

CHAPTER 3

Tables and columns

A table has a name and a list of columns. Each row of the table has one value for each column.

Creating a table

The shortest form is the name and the columns, separated by commas.

SCRIPT

```
create table student with id, name, marks, city
show tables
```

OUTPUT

```
Query OK, 0 rows affected. Table 'student' created with 4 columns.

+---------------------------+
| Tables_in_book            |
+---------------------------+
| student                   |
+---------------------------+
1 table(s) in set
```

A column with no type keeps whatever you put into it, a number or text. If you want the engine to check the kind of value, give the column a type. The columns of a table can be described afterwards with `describe`.

SCRIPT

```
create table fees with fee_id integer, student_id integer, amount integer
describe fees
```

OUTPUT

```
Query OK, 0 rows affected. Table 'fees' created with 3 columns.

+------------+---------+------+-----+---------+-------+
| Column     | Type    | Null | Key | Default | Notes |
+------------+---------+------+-----+---------+-------+
| fee_id     | integer | yes  |     |         |       |
| student_id | integer | yes  |     |         |       |
| amount     | integer | yes  |     |         |       |
+------------+---------+------+-----+---------+-------+
3 row(s) in set
```

`show columns from fees` shows the same table. The `Key` column tells you which column is the table's key. Rows are replaced when they have the same key, and

the next chapter explains what that means.

Types

Type	What it holds	Also written
`integer`	whole numbers	`int`
`real`	numbers with a point	`float`, `decimal`, `number`
`text`	text	`varchar`
`boolean`	`true` or `false`	
`date`	a date such as `2024-03-15`, with an optional time	
`timestamp`	a moment, kept in UTC: `2024-03-15 05:00:00Z`	`datetime`
`json`	text that is JSON, checked when it is written	
`blob`	bytes, written `0x0AFF`	`bytes`

Chapter 12 explains how a value is turned into the column's type, and what the engine does when it cannot be; chapter 8 has the functions for moments, JSON and bytes.

Names with spaces

A name in backticks may have spaces, and may be a word of the language.

SCRIPT

```
create table `order details` with id integer primary key, `unit price` real, `product name` text, `
    order` integer
insert into `order details` values (1, 9.5, "Chai", 3), (2, 4, "Biscuits", 10)
find `product name`, `unit price` * `order` as `line total` from `order details`
```

OUTPUT

```
Query OK, 0 rows affected. Table 'order details' created with 4 columns.

Query OK, 2 rows affected

+--------------+------------+
| product name | line total |
+--------------+------------+
| Chai         | 28.50      |
| Biscuits     | 40.00      |
+--------------+------------+
2 row(s) in set
```

Inside the backticks a name has letters, digits, _ and spaces. The engine prints it with its spaces, and so does the first line of a CSV file.

Creating a table only if it is new

The words `if not exists` stop the statement from failing when the table is already there.

SCRIPT

```
create table if not exists student with id, name
describe student
```

OUTPUT

```
Query OK, 0 rows affected. Table 'student' already exists.

+--------+------+------+-----+---------+-------+
| Column | Type | Null | Key | Default | Notes |
+--------+------+------+-----+---------+-------+
| id     | any  | yes  | id  |         |       |
| name   | any  | yes  |     |         |       |
| marks  | any  | yes  |     |         |       |
| city   | any  | yes  |     |         |       |
+--------+------+------+-----+---------+-------+
4 row(s) in set
```

The table `student` already existed, so nothing changed and its four columns are still there.

Creating a table that already exists

Without those words the result is the same: the engine leaves the table as it is, says that it already exists, and does not change its columns. It never replaces a table that has rows in it.

SCRIPT

```
create table student with id, name, email
```

OUTPUT

```
Query OK, 0 rows affected. Table 'student' already exists.
```

Looking at what you have

SCRIPT

```
show tables
describe student
```

OUTPUT

```
+--------------------------+
| Tables_in_book           |
```

```
+---------------------------+
| student                   |
| fees                      |
| order details             |
+---------------------------+
3 table(s) in set
```

```
+--------+------+------+-----+---------+-------+
| Column | Type | Null | Key | Default | Notes |
+--------+------+------+-----+---------+-------+
| id     | any  | yes  | id  |         |       |
| name   | any  | yes  |     |         |       |
| marks  | any  | yes  |     |         |       |
| city   | any  | yes  |     |         |       |
+--------+------+------+-----+---------+-------+
4 row(s) in set
```

Copying a table

`create table ... as` makes a table from the result of a query. The columns are the ones the query shows.

SCRIPT

```
insert into student with id 1, name "Asha", marks 80, city "Pune"
insert into student with id 2, name "Ravi", marks 90, city "Goa"
create table toppers as find name, marks from student where marks > 85
find * from toppers
```

OUTPUT

```
Query OK, 1 row affected

Query OK, 1 row affected

Query OK, 0 rows affected. Table 'toppers' created with 2 columns.

+------+-------+
| name | marks |
+------+-------+
| Ravi | 90    |
+------+-------+
1 row(s) in set
```

A table for one script

`create temporary table` (or `create temp table`) makes a table that lives only until the script ends. It is not saved, so the next script does not see it.

SCRIPT

```
create temporary table scratch with id
insert into scratch values (1)
```

```
find * from scratch
```

OUTPUT

```
Query OK, 0 rows affected. Table 'scratch' created with 1 columns.

Query OK, 1 row affected

+----+
| id |
+----+
| 1  |
+----+
1 row(s) in set
```

SCRIPT

```
show tables
```

OUTPUT

```
+---------------------------+
| Tables_in_book            |
+---------------------------+
| student                   |
| fees                      |
| order details             |
| toppers                   |
+---------------------------+
4 table(s) in set
```

Removing a table

Dropping a table removes its rows too, so the engine asks you to add the word `confirmed`. Without it nothing happens.

SCRIPT WITH A MISTAKE

```
drop table toppers
```

WHAT enlngdb PRINTS

```
REFUSED: dropping table 'toppers' deletes the table and all its rows.
         Nothing was changed. To go ahead, add the word 'confirmed':
         drop table toppers confirmed;

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

SCRIPT

```
drop table toppers confirmed
show tables
```

OUTPUT

```
Query OK, table 'toppers' dropped successfully.

+---------------------------+
```

```
| Tables_in_book            |
+---------------------------+
| student                   |
| fees                      |
| order details             |
+---------------------------+
3 table(s) in set
```

The same safety word appears wherever data could be lost for good: dropping a column, deleting every row, truncating a table, dropping a database. Chapter 11 lists them all.

Changing the columns later

You can add, remove and rename columns after a table exists, and change a column's type. Chapter 13 covers that.

CHAPTER 4

Adding rows

One row, written as a sentence

`insert into <table> with <column> <value>, <column> <value>, ...` adds one row. The columns you leave out get `null`, or their default if the column has one.

SCRIPT

```
create table student with id, name, marks, city
insert into student with id 1, name "Asha", marks 80, city "Pune"
insert into student with id 2, name "Ravi", marks 90, city "Goa"
find * from student
```

OUTPUT

```
Query OK, 0 rows affected. Table 'student' created with 4 columns.

Query OK, 1 row affected

Query OK, 1 row affected

+----+------+-------+------+
| id | name | marks | city |
+----+------+-------+------+
| 1  | Asha | 80    | Pune |
| 2  | Ravi | 90    | Goa  |
+----+------+-------+------+
2 row(s) in set
```

`insert record into` and `insert into` are the same.

Five ways to join a column and its value

Between a column and its value you may put nothing, a colon, an equals sign, the word `is`, or the word `to`. They all store the same thing.

SCRIPT

```
insert into student with id 3, name "Meera", marks 70
insert into student with id: 4, name: "Karan", marks: 95
insert into student with id = 5, name = "Dev", marks = 60
insert into student with id is 6, name is "Isha", marks is 85
insert into student with id to 7, name to "Omar", marks to 75
find * from student where id is at least 3
```

OUTPUT

```
Query OK, 1 row affected
```

```
Query OK, 1 row affected

Query OK, 1 row affected

Query OK, 1 row affected

Query OK, 1 row affected

+----+-------+-------+------+
| id | name  | marks | city |
+----+-------+-------+------+
| 3  | Meera | 70    | null |
| 4  | Karan | 95    | null |
| 5  | Dev   | 60    | null |
| 6  | Isha  | 85    | null |
| 7  | Omar  | 75    | null |
+----+-------+-------+------+
5 row(s) in set
```

Many rows at once

`values` takes one bracket for each row. The brackets must fit the columns of the table in the order they were made, or you can name the columns first.

SCRIPT

```
create table fees with fee_id, student_id, amount
insert into fees values (10, 1, 500), (11, 1, 300), (12, 2, 900)
insert into fees (fee_id, student_id) values (13, 9)
find * from fees
```

OUTPUT

```
Query OK, 0 rows affected. Table 'fees' created with 3 columns.

Query OK, 3 rows affected

Query OK, 1 row affected

+--------+------------+--------+
| fee_id | student_id | amount |
+--------+------------+--------+
| 10     | 1          | 500    |
| 11     | 1          | 300    |
| 12     | 2          | 900    |
| 13     | 9          | null   |
+--------+------------+--------+
4 row(s) in set
```

Several rows go in together or not at all: if one of them is refused, none is added.

Rows from a query

After the column list you can write a query instead of `values`. Its rows are added.

SCRIPT

```
create table honours with id, name
insert into honours (id, name) find id, name from student where marks >= 90
find * from honours
```

OUTPUT

```
Query OK, 0 rows affected. Table 'honours' created with 2 columns.

Query OK, 2 rows affected

+----+-------+
| id | name  |
+----+-------+
| 2  | Ravi  |
| 4  | Karan |
+----+-------+
2 row(s) in set
```

A column you did not create

If you name a column that the table does not have, the table gets that column, and the rows already there get `null` in it.

SCRIPT

```
create table events with id, event_name
insert into events with id 1, event_name "Boot"
insert into events with id 2, event_name "Disk warning", severity "high"
find * from events
```

OUTPUT

```
Query OK, 0 rows affected. Table 'events' created with 2 columns.

Query OK, 1 row affected

Query OK, 1 row affected

+----+--------------+----------+
| id | event_name   | severity |
+----+--------------+----------+
| 1  | Boot         | null     |
| 2  | Disk warning | high     |
+----+--------------+----------+
2 row(s) in set
```

This is handy in a quick script and risky in a long one, because a spelling mistake in a column name makes a new column instead of an error. Give a table its columns with `create table` when you want mistakes to be caught.

The key of a table

A row is identified by its key. The key is the column you declared as `primary key` (chapter 12), and when you declared none, a column called `id`. If you insert a row whose key is already in the table, the old row is replaced and the engine says so.

SCRIPT

```
insert into student with id 1, name "Asha", marks 82, city "Pune"
find name, marks from student where id is 1
```

OUTPUT

```
Query OK, 1 row replaced: 'student' already had a row with id 1.

+------+-------+
| name | marks |
+------+-------+
| Asha | 82    |
+------+-------+
1 row(s) in set
```

A column called `student_id` or `code` is only a column. Only the declared key, or a column named `id`, can replace a row.

Values and what they look like

- Text goes between quotes. A number written without quotes is a number.
- A text that looks like a number, such as `"80"`, is stored as the number 80 in a column whose type is `integer` or `real`. In a column without a type, it stays text.
- `null` is the value that means nothing is known.

An insert the engine refuses

When a table has a type or a rule and a value breaks it, the engine says why and adds nothing. Types and rules are in chapter 12.

SCRIPT

```
create table price_list with item text, price real not null
```

OUTPUT

```
Query OK, 0 rows affected. Table 'price_list' created with 2 columns.
```

SCRIPT WITH A MISTAKE

```
insert into price_list with item "pen"
```

WHAT enlngdb PRINTS

```
ERROR: column 'price' cannot be empty (it is marked not null).
       Nothing was inserted.

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

CHAPTER 5

Finding rows

The tables of this chapter are made once, here, and used in the chapters that follow.

SCRIPT

```
create table student with id, name, marks, city
insert into student values (1, "Asha", 80, "Pune"), (2, "Ravi", 90, "Goa"), (3, "Meera", 70, "Pune"
    ), (4, "Karan", 95, null)
```

OUTPUT

```
Query OK, 0 rows affected. Table 'student' created with 4 columns.

Query OK, 4 rows affected
```

All the rows, or some of the columns

SCRIPT

```
find * from student
find name, marks from student
```

OUTPUT

```
+----+-------+-------+------+
| id | name  | marks | city |
+----+-------+-------+------+
| 1  | Asha  | 80    | Pune |
| 2  | Ravi  | 90    | Goa  |
| 3  | Meera | 70    | Pune |
| 4  | Karan | 95    | null |
+----+-------+-------+------+
4 row(s) in set

+-------+-------+
| name  | marks |
+-------+-------+
| Asha  | 80    |
| Ravi  | 90    |
| Meera | 70    |
| Karan | 95    |
+-------+-------+
4 row(s) in set
```

`find all records from student` and `find the records from student` give the same table as `find * from student`.

Choosing rows with where

`where` keeps the rows for which a condition is true. The condition can be written in words or in marks.

Meaning	In words	As a mark
equal	`is`, `is equal to`, `equals`	`=`, `==`
not equal	`is not`, `is not equal to`	`!=`
greater	`is greater than`, `more than`, `is above`	`>`
greater or equal	`is at least`, `is greater than or equal to`	`>=`
less	`is less than`, `is under`, `below`	`<`
less or equal	`is at most`, `is less than or equal to`	`<=`

SCRIPT

```
find name from student where marks is more than 85
find name from student where marks is at most 80
find name from student where city is not "Pune"
```

OUTPUT

```
+-------+
| name  |
+-------+
| Ravi  |
| Karan |
+-------+
2 row(s) in set

+-------+
| name  |
+-------+
| Asha  |
| Meera |
+-------+
2 row(s) in set

+------+
| name |
+------+
| Ravi |
+------+
1 row(s) in set
```

Look at the last one. Karan has no city, and he is not in the answer. A comparison with `null` is never true, and neither is its opposite. To ask for the rows with no value, write `is null`:

SCRIPT

```
find name from student where city is null
```

```
find name from student where city is not null
```

OUTPUT

```
+-------+
| name  |
+-------+
| Karan |
+-------+
1 row(s) in set

+-------+
| name  |
+-------+
| Asha  |
| Ravi  |
| Meera |
+-------+
3 row(s) in set
```

and, or, not and brackets

SCRIPT

```
find name from student where city is "Pune" and marks is at least 75
find name from student where city is "Goa" or marks is more than 90
find name from student where not city is "Pune"
find name from student where (marks > 85 or marks < 75) and city is not null
```

OUTPUT

```
+------+
| name |
+------+
| Asha |
+------+
1 row(s) in set

+-------+
| name  |
+-------+
| Ravi  |
| Karan |
+-------+
2 row(s) in set

+------+
| name |
+------+
| Ravi |
+------+
1 row(s) in set

+-------+
| name  |
```

```
+-------+
| Ravi  |
| Meera |
+-------+
2 row(s) in set
```

`and` is worked out before `or`. Use brackets when you mean something else.

between, in and like

SCRIPT

```
find name from student where marks between 70 and 85
find name from student where city in ("Pune", "Goa")
find name from student where name like "A%"
find name from student where name like "_a%"
```

OUTPUT

```
+-------+
| name  |
+-------+
| Asha  |
| Meera |
+-------+
2 row(s) in set

+-------+
| name  |
+-------+
| Asha  |
| Ravi  |
| Meera |
+-------+
3 row(s) in set

+------+
| name |
+------+
| Asha |
+------+
1 row(s) in set

+-------+
| name  |
+-------+
| Ravi  |
| Karan |
+-------+
2 row(s) in set
```

`between` includes both ends. In `like`, `%` stands for any text and _ for one letter. `not between`, `not in` and `not like` ask the opposite question.

Sorting

`order by` puts the rows in order. The words `asc` or `ascending` and `desc` or `descending` choose the direction, and `highest first` and `lowest first` mean the same.

SCRIPT

```
find name, marks from student order by marks desc
find name, marks from student order by city desc, marks
find name, marks from student sorted by marks lowest first
```

OUTPUT

```
+-------+-------+
| name  | marks |
+-------+-------+
| Karan | 95    |
| Ravi  | 90    |
| Asha  | 80    |
| Meera | 70    |
+-------+-------+
4 row(s) in set

+-------+-------+
| name  | marks |
+-------+-------+
| Meera | 70    |
| Asha  | 80    |
| Ravi  | 90    |
| Karan | 95    |
+-------+-------+
4 row(s) in set

+-------+-------+
| name  | marks |
+-------+-------+
| Meera | 70    |
| Asha  | 80    |
| Ravi  | 90    |
| Karan | 95    |
+-------+-------+
4 row(s) in set
```

In `order by city desc, marks` the rows are sorted by city from the last to the first, and the rows of one city by marks. A row with `null` sorts before every value when the order is ascending.

Only some of the rows: top, first, limit, offset

SCRIPT

```
find top 2 records from student order by marks highest first
```

```
find name from student order by marks desc limit 2 offset 1
```

OUTPUT

```
+----+-------+-------+------+
| id | name  | marks | city |
+----+-------+-------+------+
| 4  | Karan | 95    | null |
| 2  | Ravi  | 90    | Goa  |
+----+-------+-------+------+
2 row(s) in set

+------+
| name |
+------+
| Ravi |
| Asha |
+------+
2 row(s) in set
```

`limit 2 offset 1` skips one row and then takes two. `top` and `first` take the first rows in the order you asked for. Always say how the rows are ordered when you cut them: without `order by` the rows come in the order they were added.

Each value once

SCRIPT

```
find distinct city from student
```

OUTPUT

```
+------+
| city |
+------+
| Pune |
| Goa  |
| null |
+------+
3 row(s) in set
```

Counting

SCRIPT

```
count records in student
count records in student where marks is at least 80
```

OUTPUT

```
+----------+
| count(*) |
+----------+
| 4        |
```

```
+----------+
1 row in set

+----------+
| count(*) |
+----------+
| 3        |
+----------+
1 row in set
```

Showing a calculated or fixed value

A query needs no table when it only calculates.

SCRIPT

```
select 6 * 7
```

OUTPUT

```
+-------+
| 6 * 7 |
+-------+
| 42    |
+-------+
1 row(s) in set
```

CHAPTER 6

Totals and groups

The tables for this chapter:

SCRIPT

```
create table student with id, name, marks, city
insert into student values (1, "Asha", 80, "Pune"), (2, "Ravi", 90, "Goa"), (3, "Meera", 70, "Pune"
    ), (4, "Karan", 95, null)
```

OUTPUT

```
Query OK, 0 rows affected. Table 'student' created with 4 columns.

Query OK, 4 rows affected
```

Totals over all the rows

A total takes a column from every row that passed `where` and gives one value.

Total	What it gives
`count(*)`	the number of rows
`count(column)`	the number of rows where the column is not null
`count(distinct column)`	the number of different values
`sum(column)`	the sum
`avg(column)`, `average(column)`	the average
`min(column)`, `max(column)`	the smallest and the largest
`total(column)`	the same as `sum`
`group_concat(column)`	the values joined into one text

SCRIPT

```
find count(*), count(city), count(distinct city) from student
find sum(marks), avg(marks), min(marks), max(marks) from student
find group_concat(name) from student
```

OUTPUT

```
+----------+-------------+----------------------+
| count(*) | count(city) | count(distinct city) |
+----------+-------------+----------------------+
| 4        | 3           | 2                    |
+----------+-------------+----------------------+
1 row(s) in set

+------------+------------+------------+------------+
```

```
| sum(marks) | avg(marks) | min(marks) | max(marks) |
+------------+------------+------------+------------+
| 335        | 83.75      | 70         | 95         |
+------------+------------+------------+------------+
1 row(s) in set

+----------------------+
| group_concat(name)   |
+----------------------+
| Asha,Ravi,Meera,Karan |
+----------------------+
1 row(s) in set
```

`count(city)` is 3 because Karan has no city. `count(*)` counts rows, whatever they hold.

One answer for each group

`group by` makes one row for each different value, and the totals are worked out for each group.

SCRIPT

```
find city, count(*), avg(marks) from student group by city
```

OUTPUT

```
+------+----------+------------+
| city | count(*) | avg(marks) |
+------+----------+------------+
| Pune | 2        | 75.00      |
| Goa  | 1        | 90.00      |
| null | 1        | 95.00      |
+------+----------+------------+
3 row(s) in set
```

Karan, with no city, forms a group of his own, shown as `null`.

Keeping some of the groups: having

`where` chooses rows before they are grouped. `having` chooses groups after the totals are worked out, so it is the place to use a total as a condition.

SCRIPT

```
find city, count(*) from student group by city having count(*) > 1
```

OUTPUT

```
+------+----------+
| city | count(*) |
+------+----------+
| Pune | 2        |
+------+----------+
```

```
1 row(s) in set
```

A total is not allowed in `where`. The engine says so instead of guessing:

SCRIPT WITH A MISTAKE

```
find city from student where count(*) > 1 group by city
```

WHAT enlngdb PRINTS

```
ERROR: count() can only be used where rows are totalled: in the select list, 'having' or 'order by'
     (not in 'where').

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

Grouping by several things

SCRIPT

```
find city, marks, count(*) from student group by city, marks
```

OUTPUT

```
+------+-------+----------+
| city | marks | count(*) |
+------+-------+----------+
| Pune | 80    | 1        |
| Goa  | 90    | 1        |
| Pune | 70    | 1        |
| null | 95    | 1        |
+------+-------+----------+
4 row(s) in set
```

Naming a result and sorting by it

`as` gives a result a name. The name can then be used in `group by` and `order by`.

SCRIPT

```
find city as c, count(*) as n, max(marks) as best from student group by c order by n desc, c
```

OUTPUT

```
+------+---+------+
| c    | n | best |
+------+---+------+
| Pune | 2 | 80   |
| null | 1 | 95   |
| Goa  | 1 | 90   |
+------+---+------+
3 row(s) in set
```

A number in `order by` means the item of that position in the list after `find`:

SCRIPT

```
find name, marks from student order by 2 desc limit 2
```

OUTPUT

```
+-------+-------+
```

```
| name  | marks |
+-------+-------+
| Karan | 95    |
| Ravi  | 90    |
+-------+-------+
2 row(s) in set
```

A column that is not grouped

In a grouped query every column should be grouped or be inside a total. When it is not, the engine shows the value from the first row of the group. That is allowed, but the value is one of several and may not be the one you meant.

Totals and a calculation

A total can be used inside a calculation, and a calculation inside a total.

SCRIPT

```
find sum(marks * 2), max(marks) - min(marks) from student
```

OUTPUT

```
+----------------+-------------------------+
| sum(marks * 2) | max(marks) - min(marks) |
+----------------+-------------------------+
| 670            | 25                      |
+----------------+-------------------------+
1 row(s) in set
```

CHAPTER 7

Joins

A join puts the rows of two tables side by side. The tables for this chapter are students, their fees, and their books.

SCRIPT

```
create table student with id, name, marks
insert into student values (1, "Asha", 80), (2, "Ravi", 90), (3, "Meera", 70)
create table fees with fee_id, student_id, amount
insert into fees values (10, 1, 500), (11, 1, 300), (12, 2, 900), (13, 9, 100)
create table book with book_id, student_id, title
insert into book values (1, 1, "Maths"), (2, 3, "Physics")
```

OUTPUT

```
Query OK, 0 rows affected. Table 'student' created with 3 columns.

Query OK, 3 rows affected

Query OK, 0 rows affected. Table 'fees' created with 3 columns.

Query OK, 4 rows affected

Query OK, 0 rows affected. Table 'book' created with 3 columns.

Query OK, 2 rows affected
```

Asha has two fees, Ravi has one and Meera none. The fee 13 belongs to student 9, who does not exist.

The kinds of join

You write the kind of join with words or with a mark. The mark is a picture: an arrow points at the table whose rows all stay.

Join	Words	Mark	What you get
inner	`join`, `inner join`	`-`	only the rows that have a match in both tables
left	`left join`, `left outer join`	`<-`	every row of the first table, with `null` where the second has no match
right	`right join`, `right outer join`	`->`	every row of the second table, with `null` where the first has no match

Join	Words	Mark	What you get
full	`full join`, `full outer join`	`<->`	every row of both tables
cross	`cross join`	`*`	every row paired with every row

Inner join

SCRIPT

```
find name, amount from student join fees on student.id is fees.student_id
```

OUTPUT

```
+------+--------+
| name | amount |
+------+--------+
| Asha | 500    |
| Asha | 300    |
| Ravi | 900    |
+------+--------+
3 row(s) in set
```

Meera has no fee and fee 13 has no student, so neither is in the answer.

Left, right and full

SCRIPT

```
find name, amount from student left join fees on student.id is fees.student_id
find name, amount from student right join fees on student.id is fees.student_id
find name, amount from student full join fees on student.id is fees.student_id
```

OUTPUT

```
+-------+--------+
| name  | amount |
+-------+--------+
| Asha  | 500    |
| Asha  | 300    |
| Ravi  | 900    |
| Meera | null   |
+-------+--------+
4 row(s) in set

+------+--------+
| name | amount |
+------+--------+
| Asha | 500    |
| Asha | 300    |
| Ravi | 900    |
| null | 100    |
+------+--------+
```

```
4 row(s) in set

+-------+--------+
| name  | amount |
+-------+--------+
| Asha  | 500    |
| Asha  | 300    |
| Ravi  | 900    |
| Meera | null   |
| null  | 100    |
+-------+--------+
5 row(s) in set
```

The left join keeps Meera and shows `null` for her amount. The right join keeps fee 13 and shows `null` for the name. The full join keeps both.

The same joins with marks

SCRIPT

```
find name, amount from student - fees
find name, amount from student <- fees
find name, amount from student -> fees
find name, amount from student <-> fees
```

OUTPUT

```
+------+--------+
| name | amount |
+------+--------+
| Asha | 500    |
| Asha | 300    |
| Ravi | 900    |
+------+--------+
3 row(s) in set

+-------+--------+
| name  | amount |
+-------+--------+
| Asha  | 500    |
| Asha  | 300    |
| Ravi  | 900    |
| Meera | null   |
+-------+--------+
4 row(s) in set

+------+--------+
| name | amount |
+------+--------+
| Asha | 500    |
| Asha | 300    |
| Ravi | 900    |
| null | 100    |
```

```
+------+--------+
4 row(s) in set

+-------+--------+
| name  | amount |
+-------+--------+
| Asha  | 500    |
| Asha  | 300    |
| Ravi  | 900    |
| Meera | null   |
| null  | 100    |
+-------+--------+
5 row(s) in set
```

Without `on` the engine matches the columns by their names. Here it finds that `student.id` and `fees.student_id` belong together, because the table `student` and the column `student_id` share a name. If it cannot find exactly one pair it stops and tells you which pairs it found and what to write after `on`.

Cross join

Every row with every row. This is rarely what you want, but it is how you make all combinations.

SCRIPT

```
find name, title from student * book
```

OUTPUT

```
+-------+---------+
| name  | title   |
+-------+---------+
| Asha  | Maths   |
| Asha  | Physics |
| Ravi  | Maths   |
| Ravi  | Physics |
| Meera | Maths   |
| Meera | Physics |
+-------+---------+
6 row(s) in set
```

Who has nothing?

A left join and `is null` find the rows that have no partner.

SCRIPT

```
find name from student <- fees where amount is null
```

OUTPUT

```
+-------+
| name  |
```

```
+--------+
| Meera |
+--------+
1 row(s) in set
```

Three tables, and then the usual clauses

SCRIPT

```
find name, amount, title from student join fees on student.id is fees.student_id join book on stude
    nt.id is book.student_id
find name, sum(amount) from student left join fees on student.id is fees.student_id group by name o
    rder by sum(amount) desc
```

OUTPUT

```
+------+--------+-------+
| name | amount | title |
+------+--------+-------+
| Asha | 500    | Maths |
| Asha | 300    | Maths |
+------+--------+-------+
2 row(s) in set

+-------+-------------+
| name  | sum(amount) |
+-------+-------------+
| Ravi  | 900         |
| Asha  | 800         |
| Meera | null        |
+-------+-------------+
3 row(s) in set
```

Naming a table for a short form

`as` gives a table a short name. This is needed to join a table with itself.

SCRIPT

```
find s.name, f.amount from student as s left join fees as f on s.id is f.student_id where f.amount
    > 400
find a.name from student a join student b on a.marks < b.marks where b.name is "Ravi"
```

OUTPUT

```
+--------+----------+
| s.name | f.amount |
+--------+----------+
| Asha   | 500      |
| Ravi   | 900      |
+--------+----------+
2 row(s) in set

+--------+
| a.name |
+--------+
| Asha   |
```

```
| Meera  |
+--------+
2 row(s) in set
```

The second query finds the students who have fewer marks than Ravi.

Same column name in both tables

`using` names a column that has the same name in both tables. Here it fails, because `student` has no `student_id` column, and the engine says so:

SCRIPT WITH A MISTAKE

```
find name, amount from student join fees using (student_id)
```

WHAT enlngdb PRINTS

```
ERROR: 'using (student_id)' needs exactly one column student_id before the join and one in fees.

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

Use `using` for tables like `fees` and `book`, which both have `student_id`:

SCRIPT

```
find amount, title from fees join book using (student_id)
```

OUTPUT

```
+--------+-------+
| amount | title |
+--------+-------+
| 500    | Maths |
| 300    | Maths |
+--------+-------+
2 row(s) in set
```

Every column of one table

SCRIPT

```
find student.*, fees.amount from student join fees on student.id is fees.student_id
```

OUTPUT

```
+------------+--------------+---------------+-------------+
| student.id | student.name | student.marks | fees.amount |
+------------+--------------+---------------+-------------+
| 1          | Asha         | 80            | 500         |
| 1          | Asha         | 80            | 300         |
| 2          | Ravi         | 90            | 900         |
+------------+--------------+---------------+-------------+
3 row(s) in set
```

Natural join

`natural join` matches every column that has the same name in both tables. Here `fees` and `book` share only `student_id`.

SCRIPT

```
find fee_id, title from fees natural join book
find fee_id, title from fees natural left join book
```

OUTPUT

```
+--------+-------+
| fee_id | title |
+--------+-------+
| 10     | Maths |
| 11     | Maths |
+--------+-------+
2 row(s) in set

+--------+-------+
| fee_id | title |
+--------+-------+
| 10     | Maths |
| 11     | Maths |
| 12     | null  |
| 13     | null  |
+--------+-------+
4 row(s) in set
```

When the tables share no column name, or when two tables before the join both have it, the engine says so instead of guessing.

SCRIPT WITH A MISTAKE

```
find name from student natural join fees
```

WHAT enlngdb PRINTS

```
ERROR: for the natural join, fees has no column with the same name as a column before it; write the
     'on' instead.

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

`student` has `id` and `name`, `fees` has `fee_id`, `student_id` and `amount`: no name in common.

CHAPTER 8

Calculations and functions

Wherever a column can be written, a calculation can be written.

SCRIPT

```
create table student with id, name, marks, city
insert into student values (1, "Asha", 80, "Pune"), (2, "Ravi", 90, "Goa"), (3, "Meera", 70, "Pune"
    ), (4, "Karan", 95, null)
```

OUTPUT

```
Query OK, 0 rows affected. Table 'student' created with 4 columns.

Query OK, 4 rows affected
```

Arithmetic

`+`, `-`, `*`, `/` and `%` (the remainder). Brackets work as in school. Write a minus with spaces, `marks - 5`: the word `marks-5` is read as one name.

SCRIPT

```
find name, marks + 5, marks * 2 - 1, marks / 4, marks % 7 from student
select 7 / 2, 7 % 2, 2 + 3 * 4, (2 + 3) * 4
```

OUTPUT

```
+-------+-----------+---------------+-----------+-----------+
| name  | marks + 5 | marks * 2 - 1 | marks / 4 | marks % 7 |
+-------+-----------+---------------+-----------+-----------+
| Asha  | 85        | 159           | 20        | 3         |
| Ravi  | 95        | 179           | 22.50     | 6         |
| Meera | 75        | 139           | 17.50     | 0         |
| Karan | 100       | 189           | 23.75     | 4         |
+-------+-----------+---------------+-----------+-----------+
4 row(s) in set

+-------+-------+-----------+-------------+
| 7 / 2 | 7 % 2 | 2 + 3 * 4 | (2 + 3) * 4 |
+-------+-------+-----------+-------------+
| 3.50  | 1     | 14        | 20          |
+-------+-------+-----------+-------------+
1 row(s) in set
```

Dividing by zero is an error, not a silent `null`:

SCRIPT WITH A MISTAKE

```
select 1 / 0
```

WHAT enlngdb PRINTS

```
ERROR: division by zero.
```

```
NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

Text

`+` adds numbers, and joins two texts. A text that is only digits is still text when it is written in quotes, so `"80" + "1"` is `801`.

SCRIPT

```
find name + "-" + city from student
select "80" + "1"
```

OUTPUT

```
+--------------------+
| name + "-" + city |
+--------------------+
| Asha-Pune          |
| Ravi-Goa           |
| Meera-Pune         |
| null               |
+--------------------+
4 row(s) in set

+------------+
| "80" + "1" |
+------------+
| 801        |
+------------+
1 row(s) in set
```

Karan has no city, so his joined text is `null`: a calculation with a missing value gives a missing value.

Text functions

Function	Result
`upper(x)`, `lower(x)`	capital or small letters
`length(x)`	the number of characters
`trim(x)`, `ltrim(x)`, `rtrim(x)`	without spaces at the ends
`substr(x, start, count)`, `substring`	a piece; the first character is number 1
`left(x, n)`, `right(x, n)`	the first or last n characters
`replace(x, old, new)`	with every `old` changed to `new`
`concat(a, b, ...)`	the texts joined; a missing value counts as empty

Function	Result
`instr(x, y)`	the place of `y` in `x`, or 0
`reverse(x)`	the characters in the opposite order

SCRIPT

```
find upper(name) as up, lower(name) as lo, length(name) as len, substr(name, 1, 2) as sub from stud
    ent
find left(name, 3) as l3, replace(name, "a", "A") as rep, instr(name, "a") as pos from student
find concat(name, "-", city) as joined, reverse(name) as back from student
```

OUTPUT

```
+-------+-------+-----+-----+
| up    | lo    | len | sub |
+-------+-------+-----+-----+
| ASHA  | asha  | 4   | As  |
| RAVI  | ravi  | 4   | Ra  |
| MEERA | meera | 5   | Me  |
| KARAN | karan | 5   | Ka  |
+-------+-------+-----+-----+
4 row(s) in set

+-----+-------+-----+
| l3  | rep   | pos |
+-----+-------+-----+
| Ash | AshA  | 4   |
| Rav | RAvi  | 2   |
| Mee | MeerA | 5   |
| Kar | KArAn | 2   |
+-----+-------+-----+
4 row(s) in set

+------------+-------+
| joined     | back  |
+------------+-------+
| Asha-Pune  | ahsA  |
| Ravi-Goa   | ivaR  |
| Meera-Pune | areeM |
| Karan-     | naraK |
+------------+-------+
4 row(s) in set
```

Number functions

Function	Result
`abs(x)`, `sign(x)`	the size, and -1, 0 or 1
`round(x)`, `round(x, places)`	rounded
`floor(x)`, `ceil(x)`, `ceiling(x)`	rounded down or up

Function	Result
`sqrt(x)`, `power(x, y)`, `pow(x, y)`, `mod(x, y)`	square root, power, remainder
`least(a, b, ...)`, `greatest(a, b, ...)`	the smallest and the largest of the values

SCRIPT

```
select abs(-4) as a, round(3.14159, 2) as r, floor(2.7) as f, ceil(2.1) as c, sqrt(81) as s, power(
    2, 10) as p
find name, least(marks, 85) as lo, greatest(marks, 85) as hi from student
find round(avg(marks), 1) from student
```

OUTPUT

```
+---+------+---+---+------+---------+
| a | r    | f | c | s    | p       |
+---+------+---+---+------+---------+
| 4 | 3.14 | 2 | 3 | 9.00 | 1024.00 |
+---+------+---+---+------+---------+
1 row(s) in set

+-------+----+----+
| name  | lo | hi |
+-------+----+----+
| Asha  | 80 | 85 |
| Ravi  | 85 | 90 |
| Meera | 70 | 85 |
| Karan | 85 | 95 |
+-------+----+----+
4 row(s) in set

+----------------------+
| round(avg(marks), 1) |
+----------------------+
| 83.80                |
+----------------------+
1 row(s) in set
```

Missing values

Function	Result
`coalesce(a, b, ...)`	the first value that is not null
`ifnull(a, b)`	`b` when `a` is null
`nullif(a, b)`	null when `a` equals `b`, else `a`
`if(condition, yes, no)`	`yes` or `no`

SCRIPT

```
find name, coalesce(city, "unknown") as c, nullif(marks, 70) as n, if(marks >= 90, "top", "ok") as
    t from student
```

OUTPUT

```
+-------+---------+------+-----+
| name  | c       | n    | t   |
+-------+---------+------+-----+
| Asha  | Pune    | 80   | ok  |
| Ravi  | Goa     | 90   | top |
| Meera | Pune    | null | ok  |
| Karan | unknown | 95   | top |
+-------+---------+------+-----+
4 row(s) in set
```

Choosing a value: case when

SCRIPT

```
find name, case when marks >= 90 then "A" when marks >= 80 then "B" else "C" end as grade from stud
    ent
```

OUTPUT

```
+-------+-------+
| name  | grade |
+-------+-------+
| Asha  | B     |
| Ravi  | A     |
| Meera | C     |
| Karan | A     |
+-------+-------+
4 row(s) in set
```

Without `else`, a row that matches no `when` gets `null`.

Dates and the clock

Function	Result
`year(d)`, `month(d)`, `day(d)`	a part of a date
`date(d)`	the date part of a date with a time
`datediff(a, b)`	the number of days from `b` to `a`
`now()`, `today()`, `current_date()`, `current_time()`	the clock of the machine
`utc_now()`	the moment now, in UTC
`hour(t)`, `minute(t)`, `second(t)`	a part of a time
`date_add(t, n, unit)`	`n` seconds, minutes, hours, days, weeks, months or years later (or earlier, for a negative `n`)

Function	Result
`seconds_between(a, b)`	the seconds from `b` to `a`, with their zones
`in_zone(t, zone)`	the moment `t` as the clock shows it in a zone: `"+05:30"`, `"UTC"`, `"local"`

SCRIPT

```
select year("2024-03-15") as y, month("2024-03-15") as m, day("2024-03-15") as d, datediff("2024-03
    -15", "2024-03-10") as days
```

OUTPUT

```
+------+---+----+------+
| y    | m | d  | days |
+------+---+----+------+
| 2024 | 3 | 15 | 5    |
+------+---+----+------+
1 row(s) in set
```

A moment can be written with its zone after it: `2024-03-15T10:30:00+05:30`. A `timestamp` column keeps it in UTC.

SCRIPT

```
create table meeting with id integer primary key, at timestamp
insert into meeting values (1, "2024-03-15T10:30:00+05:30"), (2, "2024-01-31 09:00")
select at, in_zone(at, "+05:30") as india, date_add(at, 1, "month") as later, hour(at) as h from me
    eting
```

OUTPUT

```
Query OK, 0 rows affected. Table 'meeting' created with 2 columns.

Query OK, 2 rows affected

+----------------------+--------------------------+----------------------+---+
| at                   | india                    | later                | h |
+----------------------+--------------------------+----------------------+---+
| 2024-03-15 05:00:00Z | 2024-03-15 10:30:00+05:30 | 2024-04-15 05:00:00Z | 5 |
| 2024-01-31 09:00:00Z | 2024-01-31 14:30:00+05:30 | 2024-02-29 09:00:00Z | 9 |
+----------------------+--------------------------+----------------------+---+
2 row(s) in set
```

`set time zone "+05:30"` makes a moment written without a zone be read in that zone (until then it is UTC; `local` is the computer's zone). Zones are offsets from UTC; names such as `Asia/Kolkata` are not known.

JSON and bytes

A `json` column keeps JSON text, and refuses text that is not JSON. `json_get` finds a value by a path of keys and positions.

SCRIPT

```
create table profile with id integer primary key, data json, picture blob
```

```
insert into profile values (1, '{"name": "Asha", "langs": ["hi", "en"], "age": 31}', from_base64("i
    VBORw=="))
select json_get(data, "name") as name, json_get(data, "langs[1]") as second, json_get(data, "age")
    + 1 as next_year, json_length(data, "langs") as langs, octet_length(picture) as bytes, picture
    from profile
```

OUTPUT

```
Query OK, 0 rows affected. Table 'profile' created with 3 columns.

Query OK, 1 row affected

+------+--------+-----------+-------+-------+------------+
| name | second | next_year | langs | bytes | picture    |
+------+--------+-----------+-------+-------+------------+
| Asha | en     | 32        | 2     | 4     | 0x89504E47 |
+------+--------+-----------+-------+-------+------------+
1 row(s) in set
```

`json_type` says what a value is, `json_valid` whether a text is JSON. A `blob` keeps bytes as `0x` and hex digits; `from_base64`, `to_base64`, `hex`, `unhex` and `octet_length` work on them.

Changing the type of a value

`cast(x as type)` turns a value into `integer`, `real` or `text`. When that is not possible the engine says so.

SCRIPT

```
select cast("42" as integer) + 1 as a, cast(7 as text) + "!" as b, cast(3 as real) as c
```

OUTPUT

```
+----+----+------+
| a  | b  | c    |
+----+----+------+
| 43 | 7! | 3.00 |
+----+----+------+
1 row(s) in set
```

SCRIPT WITH A MISTAKE

```
select cast("forty" as integer)
```

WHAT enlngdb PRINTS

```
ERROR: cannot turn 'forty' into an integer.

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

Calculations in conditions and sorting

SCRIPT

```
find name from student where marks % 2 is 0 and length(name) > 4
find name, marks from student order by abs(marks - 85)
```

OUTPUT

```
+-------+
| name  |
+-------+
| Meera |
+-------+
1 row(s) in set

+-------+-------+
| name  | marks |
+-------+-------+
| Asha  | 80    |
| Ravi  | 90    |
| Karan | 95    |
| Meera | 70    |
+-------+-------+
4 row(s) in set
```

The last one puts the student closest to 85 first.

CHAPTER 9

Subqueries and combining queries

A query can be used inside another query, and the answers of two queries can be put together.

SCRIPT

```
create table student with id, name, marks, city
insert into student values (1, "Asha", 80, "Pune"), (2, "Ravi", 90, "Goa"), (3, "Meera", 70, "Pune"
    ), (4, "Karan", 95, null)
create table fees with fee_id, student_id, amount
insert into fees values (10, 1, 500), (11, 1, 300), (12, 2, 900), (13, 9, 100)
```

OUTPUT

```
Query OK, 0 rows affected. Table 'student' created with 4 columns.

Query OK, 4 rows affected

Query OK, 0 rows affected. Table 'fees' created with 3 columns.

Query OK, 4 rows affected
```

A query inside brackets

A query written in brackets is a subquery. It can give a list of values, one value, or a yes or no.

A list: in

SCRIPT

```
find name from student where id in (find student_id from fees)
find name from student where id not in (find student_id from fees)
```

OUTPUT

```
+------+
| name |
+------+
| Asha |
| Ravi |
+------+
2 row(s) in set

+--------+
| name   |
+--------+
| Meera |
```

```
| Karan |
+-------+
2 row(s) in set
```

One value

A subquery that is used as a single value must give one column and at most one row. When it gives more, the engine stops and says so.

SCRIPT

```
find name, marks from student where marks > (find avg(marks) from student)
```

OUTPUT

```
+-------+-------+
| name  | marks |
+-------+-------+
| Ravi  | 90    |
| Karan | 95    |
+-------+-------+
2 row(s) in set
```

SCRIPT WITH A MISTAKE

```
find name from student where marks > (find marks from student)
```

WHAT enlngdb PRINTS

```
ERROR: a subquery used as a value gave more than one row (2 or more); add 'limit 1' or use 'in'.

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

Yes or no: exists

`exists` is true when the subquery gives at least one row. The subquery may look at the row outside it, here `student.id`, and is then worked out again for each row.

SCRIPT

```
find name from student where exists (find fee_id from fees where student_id is student.id)
find name from student where not exists (find fee_id from fees where student_id is student.id
    )
```

OUTPUT

```
+------+
| name |
+------+
| Asha |
| Ravi |
+------+
2 row(s) in set

+-------+
| name  |
+-------+
```

```
| Meera |
| Karan |
+-------+
2 row(s) in set
```

A subquery that does not look at the outside is worked out once.

A query in place of a table

A subquery after `from` makes a table for the time of the query. It needs a name.

SCRIPT

```
find t.name from (find name, marks from student where marks > 85) as t order by t.name
```

OUTPUT

```
+--------+
| t.name |
+--------+
| Karan  |
| Ravi   |
+--------+
2 row(s) in set
```

Putting two answers together

Word	Result
`union`	the rows of both, each different row once
`union all`	the rows of both, every row
`intersect`	the rows that both have
`except`	the rows of the first that the second does not have

Both queries must show the same number of columns.

SCRIPT

```
find name from student where marks > 75 union find name from student where marks < 85
find name from student where marks > 75 union all find name from student where marks < 85
find name from student where marks > 75 intersect find name from student where marks < 85
find name from student where marks > 75 except find name from student where marks < 85
```

OUTPUT

```
+-------+
| name  |
+-------+
| Asha  |
| Ravi  |
| Karan |
| Meera |
+-------+
4 row(s) in set
```

```
+-------+
| name  |
+-------+
| Asha  |
| Ravi  |
| Karan |
| Asha  |
| Meera |
+-------+
5 row(s) in set

+------+
| name |
+------+
| Asha |
+------+
1 row(s) in set

+-------+
| name  |
+-------+
| Ravi  |
| Karan |
+-------+
2 row(s) in set
```

`order by` and `limit` are written once, at the end, and apply to the whole answer.

SCRIPT

```
find name from student where marks > 85 union find name from student where marks < 75 order by name
     limit 2
```

OUTPUT

```
+-------+
| name  |
+-------+
| Karan |
| Meera |
+-------+
2 row(s) in set
```

Naming a query: with

`with` gives a name to a query that you want to use more than once, or to make a long query easier to read. Several names can be given, and each can use the ones before it.

SCRIPT

```
with big as (find name, marks from student where marks > 75) find name from big order by name
with big as (find name, marks from student where marks > 75), top as (find name from big where mark
    s > 85) find name from top
```

OUTPUT

```
+-------+
| name  |
+-------+
| Asha  |
| Karan |
| Ravi  |
+-------+
3 row(s) in set

+-------+
| name  |
+-------+
| Ravi  |
| Karan |
+-------+
2 row(s) in set
```

A query that uses itself: with recursive

`with recursive` names a query made of two parts joined by `union` or `union all`. The first part gives the starting rows. The second part uses the name, and runs again and again on the rows found in the round before, until a round finds nothing new.

SCRIPT

```
with recursive n as (select 1 as x union all find x + 1 from n where x < 5) find x from n
```

OUTPUT

```
+---+
| x |
+---+
| 1 |
| 2 |
| 3 |
| 4 |
| 5 |
+---+
5 row(s) in set
```

This is the way to walk a tree: here, everyone below a boss, at any depth.

SCRIPT

```
create table emp with id, name, boss
insert into emp values (1, "Asha", null), (2, "Ravi", 1), (3, "Meera", 2), (4, "Karan", 2), (5, "De
    v", 4)
with recursive team as (find id, name from emp where id is 2 union find e.id, e.name from emp e joi
    n team t on e.boss is t.id) find name from team order by name
```

OUTPUT

```
Query OK, 0 rows affected. Table 'emp' created with 3 columns.

Query OK, 5 rows affected
```

```
+-------+
| name  |
+-------+
| Dev   |
| Karan |
| Meera |
| Ravi  |
+-------+
4 row(s) in set
```

With `union` a row already found is not found again, so a loop in the data ends by itself. A query that keeps finding new rows for ever is stopped after 10,000 rounds, with an error, instead of running out of memory.

SCRIPT WITH A MISTAKE

```
with recursive forever as (select 1 as x union all find x + 1 from forever) find count(*) from fore
    ver
```

WHAT enlngdb PRINTS

```
ERROR: the recursive 'with forever' was stopped after 10000 rounds; make sure its second query find
    s no new rows at some point (a 'where' that stops it).

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

The older form `insert into table with column value` also starts with the word `with`. The engine tells the two apart by what follows: a name and `as` is a query, a column name and a value is a row.

CHAPTER 10

Window functions

A total, with `group by`, turns many rows into one. A window function keeps every row and adds a column worked out from other rows near it. The rows around a row are called its window, and `over (...)` describes the window.

SCRIPT

```
create table student with id, name, marks, city
insert into student values (1, "Asha", 80, "Pune"), (2, "Ravi", 90, "Goa"), (3, "Meera", 70, "Pune"
    ), (4, "Karan", 95, "Goa")
```

OUTPUT

```
Query OK, 0 rows affected. Table 'student' created with 4 columns.

Query OK, 4 rows affected
```

Ranking

SCRIPT

```
find name, marks, rank() over (order by marks desc) as place from student
```

OUTPUT

```
+-------+-------+-------+
| name  | marks | place |
+-------+-------+-------+
| Asha  | 80    | 3     |
| Ravi  | 90    | 2     |
| Meera | 70    | 4     |
| Karan | 95    | 1     |
+-------+-------+-------+
4 row(s) in set
```

`rank`, `dense_rank` and `row_number` all number the rows in the order given. They differ when two rows are equal: `rank` gives both the same number and skips the next, `dense_rank` gives both the same number and skips nothing, and `row_number` always counts 1, 2, 3 and so on.

SCRIPT

```
insert into student values (5, "Tara", 90, "Pune")
find name, marks, rank() over (order by marks desc) as r, dense_rank() over (order by marks desc) a
    s d, row_number() over (order by marks desc) as n from student
```

OUTPUT

```
Query OK, 1 row affected

+-------+-------+---+---+---+
| name  | marks | r | d | n |
```

```
+-------+-------+---+---+---+
| Asha  | 80    | 4 | 3 | 4 |
| Ravi  | 90    | 2 | 2 | 2 |
| Meera | 70    | 5 | 4 | 5 |
| Karan | 95    | 1 | 1 | 1 |
| Tara  | 90    | 2 | 2 | 3 |
+-------+-------+---+---+---+
5 row(s) in set
```

Inside groups: partition by

`partition by` splits the rows into groups, and the window is worked out inside each group.

SCRIPT

```
find name, city, row_number() over (partition by city order by marks desc) as in_city from student
```

OUTPUT

```
+-------+------+---------+
| name  | city | in_city |
+-------+------+---------+
| Asha  | Pune | 2       |
| Ravi  | Goa  | 2       |
| Meera | Pune | 3       |
| Karan | Goa  | 1       |
| Tara  | Pune | 1       |
+-------+------+---------+
5 row(s) in set
```

Totals as windows

A total with `over` gives its answer on every row.

SCRIPT

```
find name, city, sum(marks) over (partition by city) as city_total from student
find name, marks, sum(marks) over (order by id) as running_total from student
```

OUTPUT

```
+-------+------+------------+
| name  | city | city_total |
+-------+------+------------+
| Asha  | Pune | 240        |
| Ravi  | Goa  | 185        |
| Meera | Pune | 240        |
| Karan | Goa  | 185        |
| Tara  | Pune | 240        |
+-------+------+------------+
5 row(s) in set

+-------+-------+---------------+
| name  | marks | running_total |
```

```
+-------+-------+---------------+
| Asha  | 80    | 80            |
| Ravi  | 90    | 170           |
| Meera | 70    | 240           |
| Karan | 95    | 335           |
| Tara  | 90    | 425           |
+-------+-------+---------------+
5 row(s) in set
```

Without `order by` the window is the whole group. With `order by`, each row's window runs from the first row up to this one, which is why the second query gives a running total. `count`, `sum`, `avg`, `min` and `max` can all be used this way.

The row before and after: lag and lead

`lag(column)` gives the value from the row before, and `lead(column)` from the row after. A second number says how many rows away, and a third is what to show when there is no such row.

SCRIPT

```
find name, marks, lag(marks) over (order by id) as before, lead(marks, 1, 0) over (order by id) as
    after from student
```

OUTPUT

```
+-------+-------+--------+-------+
| name  | marks | before | after |
+-------+-------+--------+-------+
| Asha  | 80    | null   | 90    |
| Ravi  | 90    | 80     | 70    |
| Meera | 70    | 90     | 95    |
| Karan | 95    | 70     | 90    |
| Tara  | 90    | 95     | 0     |
+-------+-------+--------+-------+
5 row(s) in set
```

Where a window function can be used

A window function can stand in the list after `find` and in `order by`. It cannot be used in `where`, because `where` runs before the windows are worked out. To filter on a rank, put the query in brackets:

SCRIPT

```
find t.name, t.place from (find name, rank() over (order by marks desc) as place from student) as t
     where t.place <= 2
```

OUTPUT

```
+--------+---------+
| t.name | t.place |
+--------+---------+
| Ravi   | 2       |
```

```
| Karan  | 1       |
| Tara   | 2       |
+--------+---------+
3 row(s) in set
```

CHAPTER 11

Changing and deleting rows

SCRIPT

```
create table student with id, name, marks, city
insert into student values (1, "Asha", 80, "Pune"), (2, "Ravi", 90, "Goa"), (3, "Meera", 70, "Pune"
    ), (4, "Karan", 95, "Delhi")
```

OUTPUT

```
Query OK, 0 rows affected. Table 'student' created with 4 columns.

Query OK, 4 rows affected
```

Update

`update <table> set <column> = <value>` changes rows. Without `where`, every row is changed. With `where`, only the rows that match.

SCRIPT

```
update student set marks = 85 where id is 1
update student set marks = marks + 5, city = upper(city) where city is "Pune"
find * from student
```

OUTPUT

```
Query OK, 1 row(s) updated.

Query OK, 2 row(s) updated.

+----+-------+-------+-------+
| id | name  | marks | city  |
+----+-------+-------+-------+
| 1  | Asha  | 90    | PUNE  |
| 2  | Ravi  | 90    | Goa   |
| 3  | Meera | 75    | PUNE  |
| 4  | Karan | 95    | Delhi |
+----+-------+-------+-------+
4 row(s) in set
```

The new values are worked out from the old rows, and all rows are checked before any is written. If one row would break a rule, none is changed. `update records in student set ...` and `update records ...` are the same. The word `limit` stops after some rows:

SCRIPT

```
update student set marks = 0 where marks >= 90 limit 1
find name, marks from student
```

OUTPUT

```
Query OK, 1 row(s) updated.

+-------+-------+
| name  | marks |
+-------+-------+
| Asha  | 0     |
| Ravi  | 90    |
| Meera | 75    |
| Karan | 95    |
+-------+-------+
4 row(s) in set
```

The conversational form

The same change can be written as a sentence: `in <table> change <column> to <value> where ...`. After `to` the value may be written without quotes.

SCRIPT

```
in student change city to New Delhi where id is 4
find name, city from student where id is 4
```

OUTPUT

```
Query OK, 1 row(s) updated.

+-------+-----------+
| name  | city      |
+-------+-----------+
| Karan | New Delhi |
+-------+-----------+
1 row(s) in set
```

A column that does not exist

SCRIPT WITH A MISTAKE

```
update student set marx = 1 where id is 1
```

WHAT enlngdb PRINTS

```
ERROR: table 'student' has no column 'marx'. Its columns are: id name marks city.
       Nothing was changed.

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

Delete

`delete from <table> where ...` removes the rows that match.

SCRIPT

```
delete from student where id is 1 or marks between 70 and 76
find * from student
```

OUTPUT

```
Query OK, 2 row(s) deleted.

+----+-------+-------+-----------+
| id | name  | marks | city      |
+----+-------+-------+-----------+
| 2  | Ravi  | 90    | Goa       |
| 4  | Karan | 95    | New Delhi |
+----+-------+-------+-----------+
2 row(s) in set
```

`delete records from student where ...` is the same.

The safety word confirmed

A statement that changes or removes a whole table at once needs the word `confirmed` at its end. Without it the engine refuses, changes nothing, and shows what to write. `preview` (chapter 15) shows first what such a statement would do, without asking.

SCRIPT WITH A MISTAKE

```
delete all from student
```

WHAT enlngdb PRINTS

```
REFUSED: 'delete from student' with no 'where' would remove every row.
         Nothing was deleted. To go ahead, add the word 'confirmed':
         delete all from student confirmed;

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

SCRIPT

```
delete all from student confirmed
count records in student
```

OUTPUT

```
Query OK, 2 row(s) deleted.

+----------+
| count(*) |
+----------+
| 0        |
+----------+
1 row in set
```

Statement	Needs confirmed
`delete from t` with no `where` (also `delete all from t`)	yes
`update t set ...` with no `where`	yes

Statement	Needs `confirmed`
`update` or `delete` whose `where` every row meets (`where id > 0`), on a table of two rows or more	yes
`import "f.csv" into t` when rows of the file have the key of rows already there (they would be written over)	yes
`truncate table t`	yes
`drop table t`	yes
`drop column c from t`, `alter table t drop column c`	yes
`drop database d`	yes
`update` or `delete` with a `where` that leaves some rows alone, or with `limit`	no
`insert` (a row with a key already there replaces that one row, and says so)	no
`drop view v`, `drop index i`	no

`truncate table t confirmed` removes every row and keeps the table. For a table with an automatic number, it also starts counting from 1 again.

CHAPTER 12

Types and rules for columns

A column can carry rules, and the engine checks them for every row that goes in or is changed. A row that breaks a rule is refused with the reason, and nothing is written.

Writing a table with rules

The columns of `create table` can each have a type and rules after the name.

SCRIPT

```
create table person with id integer primary key auto_increment, email text unique not null, age int
    eger default 18 check (age >= 0), role text default "user", born date
describe person
```

OUTPUT

```
Query OK, 0 rows affected. Table 'person' created with 5 columns.

+--------+---------+------+---------+---------+------------------+
| Column | Type    | Null | Key     | Default | Notes            |
+--------+---------+------+---------+---------+------------------+
| id     | integer | yes  | primary |         | auto_increment   |
| email  | text    | no   | unique  |         |                  |
| age    | integer | yes  |         | 18      | check (age >= 0) |
| role   | text    | yes  |         | "user"  |                  |
| born   | date    | yes  |         |         |                  |
+--------+---------+------+---------+---------+------------------+
5 row(s) in set
```

The same list can be written in brackets and over several lines, when each line ends with an opening bracket or a comma.

SCRIPT

```
create table pet (
  id integer primary key,
  name text not null,
  weight real
)
describe pet
```

OUTPUT

```
Query OK, 0 rows affected. Table 'pet' created with 3 columns.

+--------+---------+------+---------+---------+-------+
| Column | Type    | Null | Key     | Default | Notes |
+--------+---------+------+---------+---------+-------+
| id     | integer | yes  | primary |         |       |
```

```
| name   | text    | no   |         |         |       |
| weight | real    | yes  |         |         |       |
+--------+---------+------+---------+---------+-------+
3 row(s) in set
```

The rules

Rule	What it does
`primary key`	the identity of the row; it cannot be empty, and a second row with the same key replaces the first (chapter 4)
`primary key (a, b)`	the identity is the pair of columns
`auto_increment`	a number is given when none is written: 1, then 2, and so on
`unique`	no two rows may have the same value; `unique (a, b)` for a pair
`not null`	a value must be given; `required` means the same
`default value`	the value used when none is written
`check (condition)`	the condition must be true for the row
`references table(column)`	the value must exist in that column of that table

Defaults and automatic numbers

SCRIPT

```
insert into person with email "a@x.org"
insert into person with email "b@x.org", age 30, role "admin"
find * from person
```

OUTPUT

```
Query OK, 1 row affected

Query OK, 1 row affected

+----+---------+-----+-------+------+
| id | email   | age | role  | born |
+----+---------+-----+-------+------+
| 1  | a@x.org | 18  | user  | null |
| 2  | b@x.org | 30  | admin | null |
+----+---------+-----+-------+------+
2 row(s) in set
```

A rule that is broken

SCRIPT WITH A MISTAKE

```
insert into person with email "a@x.org"
```

WHAT enlngdb PRINTS

```
ERROR: 'a@x.org' is already in column 'email' of 'person', and that column must be different in eve
    ry row.
       Nothing was inserted.

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

SCRIPT WITH A MISTAKE

```
insert into person with email "c@x.org", age -5
```

WHAT enlngdb PRINTS

```
ERROR: column 'age' does not allow this value: the check 'age >= 0' is not met.
       Nothing was inserted.

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

SCRIPT WITH A MISTAKE

```
insert into person with age 20
```

WHAT enlngdb PRINTS

```
ERROR: column 'email' cannot be empty (it is marked not null).
       Nothing was inserted.

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

Each message names the column and the rule. The first one is refused because `a@x.org` is already there, the second because `age >= 0` is false, and the third because `email` is `not null`.

Values are turned into the column's type

A value that fits the column's type, though it is written differently, is turned into that type. `"30"` in an `integer` column becomes the number 30. A value that cannot be turned is refused, with the reason.

SCRIPT

```
insert into person with email "d@x.org", age "41", born "2024-03-15"
find email, age, born from person where email is "d@x.org"
```

OUTPUT

```
Query OK, 1 row affected

+---------+-----+------------+
| email   | age | born       |
+---------+-----+------------+
| d@x.org | 41  | 2024-03-15 |
+---------+-----+------------+
```

```
1 row(s) in set
```

SCRIPT WITH A MISTAKE

```
insert into person with email "e@x.org", age "forty"
```

WHAT enlngdb PRINTS

```
ERROR: column 'age' holds whole numbers, and 'forty' is not one.
       Nothing was inserted.

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

SCRIPT WITH A MISTAKE

```
insert into person with email "f@x.org", born "2024-02-31"
```

WHAT enlngdb PRINTS

```
ERROR: column 'born' holds a date like 2024-03-15, and '2024-02-31' is not one.
       Nothing was inserted.

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

A date is checked in full, with the lengths of the months and leap years: February 31 does not exist. A column without a type keeps whatever is written. Types are checked only where they are given.

References between tables

`references table(column)` links a column to a column of another table. The engine then keeps the link true.

SCRIPT

```
create table owner with id integer primary key, name text
create table cat with id integer primary key, name text, owner_id integer references owner(id
    )
insert into owner values (1, "Asha"), (2, "Ravi")
insert into cat values (1, "Mitu", 1), (2, "Kaalu", 1)
```

OUTPUT

```
Query OK, 0 rows affected. Table 'owner' created with 2 columns.

Query OK, 0 rows affected. Table 'cat' created with 3 columns.

Query OK, 2 rows affected

Query OK, 2 rows affected
```

A cat cannot be given an owner who does not exist:

SCRIPT WITH A MISTAKE

```
insert into cat values (3, "Ghost", 9)
```

WHAT enlngdb PRINTS

```
ERROR: 9 is not in owner.id, which column 'owner_id' refers to.
       Nothing was inserted.

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

An owner whom a cat refers to cannot be deleted, and the table of owners cannot be dropped while cats refer to it:

SCRIPT WITH A MISTAKE

```
delete from owner where id is 1
```

WHAT enlngdb PRINTS

```
ERROR: 2 row(s) of 'cat' still refer to 1 in owner.id (cat.owner_id); nothing was deleted. To delet
    e them too, define the column with 'references owner(id) on delete cascade'.

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

SCRIPT WITH A MISTAKE

```
drop table owner confirmed
```

WHAT enlngdb PRINTS

```
ERROR: table 'owner' cannot be dropped: cat.owner_id refers to it.
       Drop that table first, or the column that refers to it.

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

An owner whom no cat refers to can be deleted:

SCRIPT

```
delete from owner where id is 2
find * from owner
```

OUTPUT

```
Query OK, 1 row(s) deleted.

+----+------+
| id | name |
+----+------+
| 1  | Asha |
+----+------+
1 row(s) in set
```

When the referred row is deleted: cascade or set null

`on delete cascade` after `references` says that the rows that refer to a deleted row are deleted with it. `on delete set null` empties their column instead. Without either (or with `on delete restrict`) the delete is refused, as above.

SCRIPT

```
create table house with id integer primary key, owner_id integer references owner(id) on delete cas
```

```
    cade
create table vet with id integer primary key, owner_id integer references owner(id) on delete set n
    ull
insert into owner values (3, "Meera")
insert into house values (1, 3), (2, 3)
insert into vet values (1, 3)
delete from owner where id is 3
find count(*) from house
find * from vet
```

OUTPUT

```
Query OK, 0 rows affected. Table 'house' created with 2 columns.

Query OK, 0 rows affected. Table 'vet' created with 2 columns.

Query OK, 1 row affected

Query OK, 2 rows affected

Query OK, 1 row affected

Query OK, 1 row(s) deleted; 2 row(s) that referred to them were deleted too (on delete cascade); 1
    reference(s) were set to null (on delete set null).

+----------+
| count(*) |
+----------+
| 0        |
+----------+
1 row(s) in set

+----+----------+
| id | owner_id |
+----+----------+
| 1  | null     |
+----+----------+
1 row(s) in set
```

A cascade can go on through several tables: what refers to the deleted rows goes, and what refers to those, and so on. Everything is worked out before anything changes, so if one table refuses, nothing is deleted. `set null` on a column marked `not null` refuses the delete. `on update cascade` makes the rows that refer to a key follow it when the key is changed, `on update set null` empties them, and without either a key that others refer to cannot be changed (the error says how to cascade): `references owner(id) on update cascade`.

Two columns as one key

SCRIPT

```
create table seat (row_no integer, seat_no integer, who text, primary key (row_no, seat_no))
insert into seat values (1, 1, "Asha"), (1, 2, "Ravi")
insert into seat values (1, 2, "Meera")
find * from seat
```

OUTPUT

```
Query OK, 0 rows affected. Table 'seat' created with 3 columns.

Query OK, 2 rows affected
```

```
Query OK, 1 row replaced: 'seat' already had a row with row_no 1, seat_no 2.

+--------+---------+-------+
| row_no | seat_no | who   |
+--------+---------+-------+
| 1      | 1       | Asha  |
| 1      | 2       | Meera |
+--------+---------+-------+
2 row(s) in set
```

A row with the same pair replaced the old one, and the engine said so.

CHAPTER 13

Changing the shape: alter, views, indexes

SCRIPT

```
create table student with id, name, marks, city
insert into student values (1, "Asha", 80, "Pune"), (2, "Ravi", 90, "Goa"), (3, "Meera", 70, "Pune"
    )
create table fees with fee_id, student_id, amount
insert into fees values (10, 1, 500), (11, 2, 900)
```

OUTPUT

```
Query OK, 0 rows affected. Table 'student' created with 4 columns.

Query OK, 3 rows affected

Query OK, 0 rows affected. Table 'fees' created with 3 columns.

Query OK, 2 rows affected
```

Altering a table

SCRIPT

```
alter table student add column age integer default 18
alter table student rename column name to full_name
find * from student
```

OUTPUT

```
Query OK, column 'age' added to table 'student'.

Query OK, column 'name' of 'student' is now called 'full_name'.

+----+-----------+-------+------+-----+
| id | full_name | marks | city | age |
+----+-----------+-------+------+-----+
| 1  | Asha      | 80    | Pune | 18  |
| 2  | Ravi      | 90    | Goa  | 18  |
| 3  | Meera     | 70    | Pune | 18  |
+----+-----------+-------+------+-----+
3 row(s) in set
```

Rows that were there before get the default of the new column, or `null` if there is none.

Renaming a table

SCRIPT

```
alter table student rename to pupils
```

```
show tables
```

OUTPUT

```
Query OK, table 'student' renamed to 'pupils'.

+---------------------------+
| Tables_in_book            |
+---------------------------+
| pupils                    |
| fees                      |
+---------------------------+
2 table(s) in set
```

`rename table student to pupils` is the same. The name is changed everywhere it was used, including in references from other tables.

Changing a column's type

SCRIPT

```
alter table pupils rename to student
alter table student modify column age as text
find age from student
```

OUTPUT

```
Query OK, table 'pupils' renamed to 'student'.

Query OK, column 'age' of 'student' is now text.

+-----+
| age |
+-----+
| 18  |
| 18  |
| 18  |
+-----+
3 row(s) in set
```

Every value is converted. If one value cannot be, nothing is changed, and the engine says which one.

SCRIPT WITH A MISTAKE

```
alter table student modify column full_name as integer
```

WHAT enlngdb PRINTS

```
ERROR: column 'full_name' holds whole numbers, and 'Asha' is not one; the column was not changed
    .

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

Removing a column

SCRIPT

```
alter table student drop column age confirmed
describe student
```

OUTPUT

```
Query OK, column 'age' deleted from table 'student'.

+-----------+------+------+-----+---------+-------+
| Column    | Type | Null | Key | Default | Notes |
+-----------+------+------+-----+---------+-------+
| id        | any  | yes  | id  |         |       |
| full_name | any  | yes  |     |         |       |
| marks     | any  | yes  |     |         |       |
| city      | any  | yes  |     |         |       |
+-----------+------+------+-----+---------+-------+
4 row(s) in set
```

Views

A view is a query with a name. It is kept as text, and run again each time it is used, so it always shows the rows as they are now.

SCRIPT

```
create view toppers as find full_name, marks from student where marks >= 80
find * from toppers order by full_name
update student set marks = 99 where id is 3
find * from toppers order by full_name
show views
```

OUTPUT

```
Query OK, view 'toppers' created.

+-----------+-------+
| full_name | marks |
+-----------+-------+
| Asha      | 80    |
| Ravi      | 90    |
+-----------+-------+
2 row(s) in set

Query OK, 1 row(s) updated.

+-----------+-------+
| full_name | marks |
+-----------+-------+
| Asha      | 80    |
| Meera     | 99    |
| Ravi      | 90    |
+-----------+-------+
3 row(s) in set

+---------+-----------------------------------------------------+
```

```
| View    | Query                                                  |
+---------+--------------------------------------------------------+
| toppers | find full_name, marks from student where marks >= 80 |
+---------+--------------------------------------------------------+
1 row(s) in set
```

After the update Meera belongs in the view without anyone changing the view. Rows cannot be inserted into a view.

SCRIPT WITH A MISTAKE

```
insert into toppers values ("Zed", 100)
```

WHAT enlngdb PRINTS

```
ERROR: 'toppers' is a view, and rows cannot be inserted into a view.
       Nothing was inserted.

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

`create or replace view` changes a view, and `drop view toppers` removes it, without `confirmed` because no rows are lost.

SCRIPT

```
create or replace view toppers as find full_name from student where marks = 99
find * from toppers
drop view toppers
```

OUTPUT

```
Query OK, view 'toppers' saved.

+-----------+
| full_name |
+-----------+
| Meera     |
+-----------+
1 row(s) in set

Query OK, view 'toppers' dropped.
```

Indexes

SCRIPT

```
create unique index u_name on student (full_name)
create index by_city on student (city)
show indexes from student
```

OUTPUT

```
Query OK, unique index 'u_name' made on student (full_name).

Query OK, index 'by_city' made on student (city).
       Note: EnlngDB makes a lookup index by itself when a query needs one, so this index is only a
     record.
```

```
+---------+---------+-----------+--------+
| Table   | Index   | Columns   | Unique |
+---------+---------+-----------+--------+
| student | (id)    | id        | yes    |
| student | u_name  | full_name | yes    |
| student | by_city | city      | no     |
+---------+---------+-----------+--------+
3 row(s) in set
```

A `unique` index is a rule: two rows may not have the same value in it. A plain index is written down and shown by `show indexes`, but it does not make queries faster. EnlngDB keeps every table in memory and reads it in full for each query. `explain` says this about any query.

SCRIPT

```
drop index by_city
explain find full_name from student join fees on student.id is fees.student_id order by full_name l
    imit 2
```

OUTPUT

```
Query OK, index 'by_city' dropped.

  1. read every row of table student (3 rows)
  2. inner join table fees (2 rows), matching the 'on' condition
  3. sort the rows (1 key)
  4. keep only the rows asked for with top / limit / offset
(EnlngDB reads tables from memory. 'where column is value' on a table of 48 rows or more uses a loo
    kup index it makes by itself.)
```

Looking at a table

`describe student` and `show columns from student` list the columns, their types, whether they may be empty, which are keys, and their defaults and rules.

SCRIPT

```
describe student
```

OUTPUT

```
+-----------+------+------+-----+---------+-------+
| Column    | Type | Null | Key | Default | Notes |
+-----------+------+------+-----+---------+-------+
| id        | any  | yes  | id  |         |       |
| full_name | any  | yes  |     |         |       |
| marks     | any  | yes  |     |         |       |
| city      | any  | yes  |     |         |       |
+-----------+------+------+-----+---------+-------+
4 row(s) in set
```

CHAPTER 14

Transactions and databases

Taking a change back

A transaction makes a group of changes that you can undo. `begin` starts it. `commit` keeps every change since `begin`. `rollback` throws them all away.

SCRIPT

```
create table account with id integer primary key, owner text, balance integer
insert into account values (1, "Asha", 500), (2, "Ravi", 300)
begin
update account set balance = balance - 200 where id is 1
update account set balance = balance + 200 where id is 2
find * from account
rollback
find * from account
```

OUTPUT

```
Query OK, 0 rows affected. Table 'account' created with 3 columns.

Query OK, 2 rows affected

Query OK, transaction started. 'commit' keeps what follows, 'rollback' undoes it.

Query OK, 1 row(s) updated.

Query OK, 1 row(s) updated.

+----+-------+---------+
| id | owner | balance |
+----+-------+---------+
| 1  | Asha  | 300     |
| 2  | Ravi  | 500     |
+----+-------+---------+
2 row(s) in set

Query OK, transaction rolled back: everything since 'begin' is undone.

+----+-------+---------+
| id | owner | balance |
+----+-------+---------+
| 1  | Asha  | 500     |
| 2  | Ravi  | 300     |
+----+-------+---------+
2 row(s) in set
```

After `rollback` the balances are what they were. With `commit` in its place, they would be kept.

SCRIPT

```
begin
update account set balance = balance - 200 where id is 1
update account set balance = balance + 200 where id is 2
commit
find * from account
```

OUTPUT

```
Query OK, transaction started. 'commit' keeps what follows, 'rollback' undoes it.

Query OK, 1 row(s) updated.

Query OK, 1 row(s) updated.

Query OK, transaction committed.

+----+-------+---------+
| id | owner | balance |
+----+-------+---------+
| 1  | Asha  | 300     |
| 2  | Ravi  | 500     |
+----+-------+---------+
2 row(s) in set
```

A transaction covers the whole database: its tables, their rows, and its views. Anything inside it can be undone, including `create table` and `drop table`. `begin` copies nothing: the transaction keeps records of what each statement in it changed, so it costs what the changes cost, also on a big database.

A transaction that is never closed

If a script ends while a transaction is open, the engine rolls it back and says so, so that a forgotten `commit` never keeps half of a change.

SCRIPT

```
begin
delete from account where id is 1
```

OUTPUT

```
Query OK, transaction started. 'commit' keeps what follows, 'rollback' undoes it.

Query OK, 1 row(s) deleted.

NOTE: a transaction was still open at the end, so it was rolled back (add 'commit' to keep it).
```

SCRIPT

```
find * from account
```

OUTPUT

```
+----+-------+---------+
| id | owner | balance |
+----+-------+---------+
```

```
| 1  | Asha  | 300     |
| 2  | Ravi  | 500     |
+----+-------+---------+
2 row(s) in set
```

Asha is still there.

Savepoints

A savepoint is a name for a point inside a transaction. `rollback to` goes back to it and keeps the transaction open; `release` forgets it.

SCRIPT

```
begin
update account set balance = 100 where id is 1
savepoint before_ravi
update account set balance = 0 where id is 2
rollback to before_ravi
commit
find * from account
```

OUTPUT

```
Query OK, transaction started. 'commit' keeps what follows, 'rollback' undoes it.

Query OK, 1 row(s) updated.

Query OK, savepoint 'before_ravi' made. 'rollback to before_ravi' goes back to here.

Query OK, 1 row(s) updated.

Query OK, back at savepoint 'before_ravi': everything after it is undone. The transaction is still
    open.

Query OK, transaction committed.

+----+-------+---------+
| id | owner | balance |
+----+-------+---------+
| 1  | Asha  | 100     |
| 2  | Ravi  | 500     |
+----+-------+---------+
2 row(s) in set
```

The change to Asha is kept; the change to Ravi was taken back. A transaction can have 16 savepoints.

One program at a time

While a script has a database open, the operating system holds a lock on the file `<name>.edb.lock`. A second program that opens the same database waits up to 5 seconds and then stops with an error that says the database is in use. The lock goes away when the first program ends, also when it crashes. For several programs at once there is the server, below.

Users and rights

A user has a name and a password of 8 characters or more; only a salted hash of the password is kept in the file. An admin may do everything. Other users may do what they were granted: `select`, `insert`, `update` and `delete` on a table (or on `*`, every table), and `execute` on a procedure.

SCRIPT

```
create user boss password "boss-pass-1" admin
create user asha password "asha-pass-1"
grant select, update on account to asha
show users
show grants for asha
revoke update on account from asha
show grants for asha
```

OUTPUT

```
Query OK, user 'boss' made (an admin: may do everything). Only a salted hash of the password is kep
    t.

Query OK, user 'asha' made; grant it what it may do. Only a salted hash of the password is kept.

Query OK, asha may now select, update on account.

+------+-------+
| User | Admin |
+------+-------+
| boss | yes   |
| asha | no    |
+------+-------+
2 row(s) in set

+------+---------+----------------+
| User | On      | May            |
+------+---------+----------------+
| asha | account | select, update |
+------+---------+----------------+
1 row(s) in set

Query OK, asha may no longer update on account (unless another grant allows it).

+------+---------+--------+
| User | On      | May    |
+------+---------+--------+
| asha | account | select |
+------+---------+--------+
1 row(s) in set
```

Rights are checked by the server. A program that opens the file itself, like every script in this book, is its owner, and nothing is checked: whoever can read the file can read the data.

The server

`enlngdb --serve` keeps databases open and lets many clients sign in at the same time, from this computer or another one. A client says which database it

signs in to after a slash.

TERMINAL

```
enlngdb --serve bank shop --port 7171
enlngdb --connect 7171/bank --user asha report.enlngdb
```

The client asks for the password without showing it (or reads it from `ENLNGDB_PASSWORD`), signs in, runs the script and prints what the server answered. Without a script it runs each line typed. For a user who is not an admin, the server refuses a statement that reads a table the user may not `select`, counting every table named in the statement, in a join, a subquery or `with`, and also a name in quotes. It refuses an insert, update or delete without that right on the table it changes, and anything that makes, changes or drops tables, users or rights. `import`, `export`, `backup` and `restore` are for an admin, because they use files on the server's computer. Views, triggers and procedures run with the rights of the admin who made them. Requests that change something run one at a time, and their answer is sent when the change is on the disk: requests of several clients that arrive together are written by one save, and if the save fails each of them is told so. Requests that only read (`find`, `select`, `show`, `count` ...) go to reader processes, which answer at the same time: a slow query of one client does not hold up the lookups of the others. A reader opens the saved file, and gets a request only when every change before it is saved and no transaction is open. `--readers N` says how many readers there are (`--readers 0`: none). While a client has a transaction open, the others wait for it, up to 10 seconds. A client that goes away with a transaction open has it rolled back. The database is saved after each change that no transaction holds back, and when the server stops: Ctrl+C, or an admin running `shutdown server`. `undo` takes back only the same client's change. The connection is encrypted. The server prints the fingerprint of its key when it starts; the first time a client connects, it shows the fingerprint it got and asks whether it is the same (or takes it from `--trust`), and from then on it refuses a server at that address with a different key. Every message is sealed with ChaCha20-Poly1305, with keys agreed by X25519 for that connection, so it cannot be read or changed on the way. By default the server listens only on the computer it runs on; `--host 0.0.0.0` lets other computers in. Three wrong passwords end a connection, and after five from one address that address must wait (30 seconds, then longer each time, up to an hour) before it may sign in again. A connection that does not sign in within 30 seconds is closed, so connections that only wait cannot fill the server.

Databases

A database is a set of tables, saved as one file called `<name>.edb`. `create database` makes a new one and switches to it, and `use database` switches to one that exists.

SCRIPT

```
show databases
create database shop
create table item with id, title
insert into item values (1, "Pen")
use database book
show tables
```

OUTPUT

```
+----------+------+-----------+
| Database | Open | Size      |
+----------+------+-----------+
| book     | yes  | 1.1 KB    |
+----------+------+-----------+
1 row(s) in set

Query OK, database 'shop' created and now in use.

Query OK, 0 rows affected. Table 'item' created with 2 columns.

Query OK, 1 row affected

Database changed to 'book'.

+---------------------------+
| Tables_in_book            |
+---------------------------+
| account                   |
+---------------------------+
1 table(s) in set
```

`show databases` lists only the database that is open. The files of the others are in the folder, and `use database shop` opens `shop.edb`.

SCRIPT

```
use database shop
find * from item
```

OUTPUT

```
Database changed to 'shop'.

+----+-------+
| id | title |
+----+-------+
| 1  | Pen   |
```

```
+----+-------+
1 row(s) in set
```

Removing a database

`drop database` deletes the database and its file, and needs `confirmed`.

SCRIPT WITH A MISTAKE

```
drop database shop
```

WHAT enlngdb PRINTS

```
REFUSED: dropping database 'shop' deletes all its tables and rows.
         Nothing was changed. To go ahead, add the word 'confirmed':
         drop database shop confirmed;

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

SCRIPT

```
drop database shop confirmed
```

OUTPUT

```
Query OK, database 'shop' dropped.
```

Backup and restore

`backup database to "file.edb"` writes a whole copy of the open database to a file of its own: every table and row (with the changes of the script so far), the views, indexes, triggers, procedures and users. The database and its own file are not touched. The copy is a database file like any other, so it can be opened (`use database shop_monday` for `shop_monday.edb`), and a copy of an encrypted database is encrypted with the same password.

SCRIPT

```
backup database to "shop_monday.edb"
```

OUTPUT

```
Query OK, database 'book' backed up to 'shop_monday.edb': 1 table(s), 2 row(s).
       To bring it back: restore database from "shop_monday.edb" confirmed
```

When the file is there already, the backup asks for `confirmed`, because it would write over it. A backup never goes to the database's own file or its log. `restore database from "file.edb" confirmed` makes the open database what the file holds, and needs `confirmed` because everything in the database is replaced. The file is read and checked first: a file that is not there, is not a database, or is encrypted without the password (`with password "..."` before `confirmed`) changes nothing. The restored database is written to its file at the end of the

script, as after any change. A restore cannot be taken back with `undo`; make a backup before it if what is there now matters.

TERMINAL

```
restore database from "shop_monday.edb" confirmed
```

Inside a transaction neither runs: `commit` or `rollback` first. On a server only an admin may back up or restore, because they write and read files on the server's computer.

An encrypted database

`encrypt database with password "..."` makes the engine write the file encrypted from then on (ChaCha20-Poly1305, with a key made from the password by PBKDF2). Without the password the file cannot be read.

TERMINAL

```
enlngdb shop_setup.enlngdb                  (it says: encrypt database with password "a long passph
    rase")
set ENLNGDB_DB_PASSWORD=a long passphrase
enlngdb shop_report.enlngdb
```

A script can also say `use database shop with password "a long passphrase"`, and `enlngdb --serve` asks for the password or takes it from `ENLNGDB_DB_PASSWORD`. Without it, or with a wrong one, nothing runs on the database and the file is left as it is; a file that was changed is refused. `decrypt database` writes it plain again. A lost password cannot be found again.

CHAPTER 15

More than SQL: files, preview, undo, speed

This chapter is about things a SQL database usually leaves to other tools, or does not have at all.

SCRIPT

```
create table pupil with name text, roll_no integer primary key, class integer, stream text
insert into pupil values ("Asha", 1, 11, "Science"), ("Ravi, Jr", 2, 12, "Commerce"), ("Sita", 3, n
    ull, "Arts"), ("Karan", 4, 12, "Science")
```

OUTPUT

```
Query OK, 0 rows affected. Table 'pupil' created with 4 columns.

Query OK, 4 rows affected
```

Writing a table to a CSV file

`export` writes a table, or the result of a query, to a CSV file that a spreadsheet can open. The first line names the columns.

SCRIPT

```
export pupil to "pupils.csv"
export find name, class from pupil where stream is not "Arts" to "science.csv"
```

OUTPUT

```
Query OK, 4 rows written to 'pupils.csv'.

Query OK, 3 rows written to 'science.csv'.
```

Text that has a comma or a quote in it is put in quotes, and so is text that would otherwise read back as a number, `true`, `false` or nothing, such as `"12"`. An empty value is `null`. The file is written to a temporary file first and then moved into place.

Reading a CSV file

`import` reads a CSV file into a table. A table that does not exist is made from the first line.

SCRIPT

```
import "pupils.csv" into pupil_copy
find * from pupil_copy
```

OUTPUT

```
Query OK, 4 rows imported into 'pupil_copy' from 'pupils.csv' (a new table).

+----------+---------+-------+----------+
| name     | roll_no | class | stream   |
+----------+---------+-------+----------+
| Asha     | 1       | 11    | Science  |
| Ravi, Jr | 2       | 12    | Commerce |
| Sita     | 3       | null  | Arts     |
| Karan    | 4       | 12    | Science  |
+----------+---------+-------+----------+
4 row(s) in set
```

A value in quotes is text; without quotes it is a number, `true`, `false`, or, when it is empty, `null`. When the table has types and rules, they are checked for every row, as for `insert`. All rows go in, or none: the engine names the line that is wrong.

SCRIPT

```
create table strict_pupil with name text not null, roll_no integer primary key, class integer, stre
    am text
```

OUTPUT

```
Query OK, 0 rows affected. Table 'strict_pupil' created with 4 columns.
```

SCRIPT WITH A MISTAKE

```
import "science.csv" into strict_pupil
```

WHAT enlngdb PRINTS

```
ERROR: line 2 of 'science.csv' was refused: column 'roll_no' is the primary key of 'strict_pupil',
    so it cannot be empty.
       Nothing was imported.

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

The file `science.csv` has only the columns `name` and `class`. `roll_no` is the primary key of `strict_pupil`, and a key cannot be empty, so the first row is refused and nothing is imported. A line of the wrong length is reported with its line number, and so is a column the table does not have.

Seeing a change before making it

`preview` in front of a statement runs it on a copy of the database, shows what it would delete, change or add, and keeps nothing.

SCRIPT

```
preview delete from pupil where class is null
preview update pupil set class = 11 where roll_no is 4
count records in pupil
```

OUTPUT

```
PREVIEW of: delete from pupil where class is null

Query OK, 1 row(s) deleted.

would delete 1 row(s) of 'pupil':
+------+---------+-------+--------+
| name | roll_no | class | stream |
+------+---------+-------+--------+
| Sita | 3       | null  | Arts   |
+------+---------+-------+--------+
1 row(s) in set

PREVIEW only: nothing was changed. Run it without 'preview' to do it.

PREVIEW of: update pupil set class = 11 where roll_no is 4

Query OK, 1 row(s) updated.

would change 1 row(s) of 'pupil':
+-------+---------+-------+---------+
| name  | roll_no | class | stream  |
+-------+---------+-------+---------+
| Karan | 4       | 12    | Science |
+-------+---------+-------+---------+
1 row(s) in set

  ... into these 1 row(s) of 'pupil':
+-------+---------+-------+---------+
| name  | roll_no | class | stream  |
+-------+---------+-------+---------+
| Karan | 4       | 11    | Science |
+-------+---------+-------+---------+
1 row(s) in set

PREVIEW only: nothing was changed. Run it without 'preview' to do it.

+----------+
| count(*) |
+----------+
| 4        |
+----------+
1 row in set
```

The count after the previews is still 4.

Taking a change back

`undo` takes back the last `update`, `delete`, `truncate`, `alter`, `drop` or `import`, once.

SCRIPT

```
delete from pupil where stream is "Science"
```

```
find name from pupil
undo
find name from pupil
```

OUTPUT

```
Query OK, 2 row(s) deleted.

+----------+
| name     |
+----------+
| Ravi, Jr |
| Sita     |
+----------+
2 row(s) in set

Query OK, undone: delete from pupil where stream is "Science"

+----------+
| name     |
+----------+
| Asha     |
| Ravi, Jr |
| Sita     |
| Karan    |
+----------+
4 row(s) in set
```

`undo` works only while nothing else has changed after that statement; an `insert` after a `delete` means the `delete` can no longer be undone, because going back would lose the insert. It lives for as long as the script runs.

SCRIPT WITH A MISTAKE

```
undo
```

WHAT enlngdb PRINTS

```
ERROR: there is nothing to undo. 'undo' takes back the last update, delete, truncate, alter, drop o
    r import, as long as nothing else changed after it.

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

Natural join, savepoints, cascades, recursion

These are described with the rest of their kind: `natural join` in chapter 7, `on delete cascade` and `on delete set null` in chapter 12, savepoints in chapter 14. A recursive `with` is in chapter 9.

How the engine stays fast

Where a statement looks rows up by value, the engine builds a hash for that statement: for the key, `unique` columns and references when rows are inserted,

for a join on `a.x = b.y`, for `in (subquery)`, `union`, `distinct`, `intersect`, `except` and a recursive `with`. The answer is the same as when every row is compared: `1`, `1.0` and `"1"` are equal, text is compared without case, and a number with a fraction is compared one by one. These times were measured on one machine with `tests/bench_enlngdb.py` and 20,000 rows, before and after the hashes were added:

Work	Before	After
join 20,000 x 20,000 rows on a key	234 s	0.6 s
insert 20,000 rows with `unique` and `auto_increment`	15.1 s	0.25 s
insert 20,000 rows that refer to another table	5.9 s	0.5 s
`in (subquery)` over 20,000 rows	4.4 s	0.4 s
insert 20,000 rows in one statement	3.4 s	0.4 s
5,000 inserts, one statement each	1.9 s	0.3 s
2,000 queries `where id is X` on 20,000 rows	19.3 s	0.34 s
2,000 updates `where id is X` on 20,000 rows	24.0 s	0.15 s
500 inserts, one statement each, into 100,000 rows	0.80 s	0.25 s
100 deletes `where id is X` on 200,000 rows	4.74 s	0.46 s
300 inserts into one table, each followed by `where id is X` on another of 200,000 rows	12.4 s	0.54 s
200 queries `where id between a and b` on 200,000 rows	26.8 s	0.58 s
200 queries `where id > a and id < b` on 200,000 rows	41.5 s	0.63 s
200 updates on 200,000 rows, each taken back with `undo`	refused	0.46 s
300 queries `where name is n7` (a bare word) on 200,000 rows	53.6 s	0.58 s
300 queries `where id is a or id is b` on 200,000 rows	55.4 s	0.52 s
300 queries `where name like "n1999%"` on 200,000 rows	82.4 s	1.6 s
300 queries `where name like "%377%"` on 200,000 rows	91.4 s	1.05 s

The lines with 200,000 rows include opening them, about 0.4 s. "Refused" means `undo` said the database was too big for it. The engine makes two kinds of index by itself, the first time a query needs one, on a table of 48 rows or more. A condition `column is value` (alone, or joined to others with `and`) uses a hash of the column. A condition `column > value`, `<`, `>=`, `<=` or `column between a and b` uses a sorted list of the column: numbers and texts are kept in two lists, because they do not sort together (`"10"` and `"9"` compare as texts with each other and as numbers with a number), so a bound that is a number reads its range of numbers and every text, and the other way round. An index is kept while the table does not change, through updates of other columns, through inserts, through deletes (the other rows only move up) and through `undo`, and while other tables change; it is made again after an alter or a change of its column. The whole `where` is still tested on the rows found, so the answer is the same as when every row is read; when a range would take most of the rows, the rows are read in order instead. So there is no need to write `create index` for speed: a plain `create index` is only recorded, a `unique` one is checked. A value can be a bare word, as in `where name is Ravi`. `a or b` uses the indexes when both sides can, and joins the rows they give. `column like "abc%"` (a fixed start, then `%` or an underscore) reads the texts that start that way, and every number. Any other `like` (`"%dra%"`, `"%sha"`, or `"dra"`, which looks anywhere) uses a third index, made of every three letters of every text: it reads the rows that have every three letters of the pattern's longest part between the wildcards, and every number; a pattern whose longest part is shorter than three letters reads every row. So do `not` (most rows pass it anyway) and a condition on a calculation, like `id + 0 = 5`. How big can a database be? On one machine a table of an `id`, a text and a number with 1,000,000 rows is a 38 MB file and takes about 180 MB of memory when every row is read; a script opens it and answers a query in about 0.4 s.

A file is opened without reading its rows: each row is read the first time a statement needs it, and the file keeps a lookup index of the key, of the `unique` columns and of one-column indexes. So a script that finds a few rows by such a column reads only those. The file keeps those columns in order too, so `id > 1000`, `between`, and `name like "abc%"` on them read only the rows of the range: with 300,000 rows, a range of 41 keys took 0.03 s and 5.7 MB, where reading every row took 0.09 s and 37 MB. With 3,000,000 rows (a 262 MB file), opening it and finding a row by its key took 0.2 s and 49 MB, where reading every row took 5.2 s and 1.1 GB. A query that needs every row still reads them

all, and a script that changes the database writes only the change at the end, to a small log beside the file (`shop.edb-wal`): changing one row of those 3,000,000 and saving took 0.1 s, where writing the whole file took 6.3 s. The whole file is written only now and then: when the log grows past a quarter of the file, or when columns or tables change.

It can be larger than memory. Up to a budget (half of the computer's memory, or what `ENLNGDB_MEMORY` says, such as `500M`) the engine takes memory as usual; past it, the rows go into a file that the system keeps in memory only where it is used. The answers are the same; past the budget the work goes at the speed of the disk. With the memory of the program limited to 300 MB, 3,000,000 rows (about 1 GB in memory) did not fit; with `ENLNGDB_MEMORY=100M` they were imported in 37 s, and opened, changed and saved in 25 s, using at most 150 MB.

TERMINAL

```
set ENLNGDB_MEMORY=100M
set ENLNGDB_MEMSTAT=1
enlngdb big_report.enlngdb
```

`ENLNGDB_MEMSTAT=1` prints at the end how much was in memory and how much in the file. What it does not do: the log is for rows; a change of columns, tables, triggers or users writes the whole file, and an encrypted database is always written whole. If the log is damaged (its checksum does not fit), the database is not opened and nothing is written, so no change is lost without a word; moving the log away opens the file as it was at its last full write. `undo` does not copy the tables. It keeps only what the statement changed: the old values of the rows an update changed, the rows a delete took away, the number of rows an import added, a copy of a table whose columns an alter changed. So it works on a database of any size, and costs about what the change itself cost. Undoing a `drop table` brings its triggers back too.

CHAPTER 16

Triggers, procedures and helpful errors

A trigger is a rule that runs by itself when rows change. A procedure is a few statements saved under a name. This chapter has both, and ends with how the engine helps when a word is mistyped.

SCRIPT

```
create table account with id integer primary key, owner text, balance integer
create table ledger with id integer primary key auto_increment, what text, account_id integer, amou
    nt integer
insert into account values (1, "Asha", 100), (2, "Ravi", 50)
```

OUTPUT

```
Query OK, 0 rows affected. Table 'account' created with 3 columns.

Query OK, 0 rows affected. Table 'ledger' created with 4 columns.

Query OK, 2 rows affected
```

A trigger that keeps a record

`create trigger` says when it runs (`after insert`, `after update` or `after delete`), on which table, and what it does after `do`. Inside it, `new.balance` is the value after the change and `old.balance` the value before.

SCRIPT

```
create trigger log_change after update on account when new.balance is not old.balance do insert int
    o ledger (what, account_id, amount) values ("balance changed", new.id, new.balance - old.balanc
    e)
update account set balance = balance - 30 where id is 1
update account set owner = "Ravi K" where id is 2
find what, account_id, amount from ledger
```

OUTPUT

```
Query OK, trigger 'log_change' made: after update on account. In it, new.column and old.column are
    the row's values.

Query OK, 1 row(s) updated.

Query OK, 1 row(s) updated.

+-----------------+------------+--------+
| what            | account_id | amount |
+-----------------+------------+--------+
| balance changed | 1          | -30    |
+-----------------+------------+--------+
1 row(s) in set
```

The first update changed a balance, so the trigger ran once, for that row. The second changed a name only, so `when new.balance is not old.balance` was not true and nothing was written. The trigger runs once for each row that changed, also for rows changed by `import` or by a cascade.

A trigger that refuses

`raise "message"` stops a statement. When a trigger raises an error, or one of its statements fails, the whole statement is taken back, with everything the other triggers did.

SCRIPT

```
create trigger no_overdraft after update on account when new.balance < 0 do raise "the balance cann
    ot go below zero"
```

OUTPUT

```
Query OK, trigger 'no_overdraft' made: after update on account. In it, new.column and old.column ar
    e the row's values.
```

SCRIPT WITH A MISTAKE

```
update account set balance = balance - 500 where id is 2
```

WHAT enlngdb PRINTS

```
ERROR: the trigger 'no_overdraft' stopped this statement: the balance cannot go below zero.
       Nothing was changed.

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

SCRIPT

```
find owner, balance from account
count records in ledger
```

OUTPUT

```
+--------+---------+
| owner  | balance |
+--------+---------+
| Asha   | 70      |
| Ravi K | 50      |
+--------+---------+
2 row(s) in set

+----------+
| count(*) |
+----------+
| 1        |
+----------+
1 row in set
```

Ravi's balance is still 50 and the ledger has no extra row.

Before triggers

A trigger made with `before` runs for each row before the change counts. `set new.column = value` changes the row that is being added or changed, and `raise "message"` stops the statement. Each `set` sees the values that the ones before it gave.

SCRIPT

```
create trigger tidy before insert on account do set new.owner = upper(new.owner)
insert into account values (3, "meera", 0)
find id, owner from account where id is 3
```

OUTPUT

```
Query OK, trigger 'tidy' made: before insert on account. In it, new.column and old.column are the r
    ow's values.

Query OK, 1 row affected

+----+-------+
| id | owner |
+----+-------+
| 3  | MEERA |
+----+-------+
1 row(s) in set
```

The row a before trigger changed must still keep every rule of the table, or the statement is taken back. A before trigger cannot change a column that another table refers to, and a `before delete` trigger has no new row to set. Its other statements can change other tables. They run once the rows of the statement are settled, before the after triggers, and they set off the triggers of the tables they change.

SCRIPT

```
create trigger note_new before insert on account do insert into ledger (what, account_id, amount) v
    alues ("opened", new.id, new.balance)
insert into account values (4, "kiran", 25)
find what, account_id, amount from ledger where what is "opened"
drop trigger note_new
```

OUTPUT

```
Query OK, trigger 'note_new' made: before insert on account. In it, new.column and old.column are t
    he row's values.

Query OK, 1 row affected

+--------+------------+--------+
| what   | account_id | amount |
+--------+------------+--------+
| opened | 4          | 25     |
+--------+------------+--------+
1 row(s) in set

Query OK, trigger 'note_new' dropped.
```

A trigger for a whole statement

With `for each statement`, a trigger runs once after a statement that changed rows of its table, however many. `count.rows` is how many.

SCRIPT

```
create table stock with id integer primary key, qty integer
create table stock_log with what text, rows_changed integer
insert into stock values (1, 5), (2, 0), (3, 7)
create trigger restock after update on stock for each statement do insert into stock_log values ("u
    pdate", count.rows)
update stock set qty = qty + 10 where qty < 6
select what, rows_changed from stock_log
```

OUTPUT

```
Query OK, 0 rows affected. Table 'stock' created with 2 columns.

Query OK, 0 rows affected. Table 'stock_log' created with 2 columns.

Query OK, 3 rows affected

Query OK, trigger 'restock' made: after update on stock, once for each statement that updates rows.
     In it, count.rows is how many.

Query OK, 2 row(s) updated.

+--------+--------------+
| what   | rows_changed |
+--------+--------------+
| update | 2            |
+--------+--------------+
1 row(s) in set
```

A statement that changes no row does not set it off. It runs after the rows, so it is always `after`.

Seeing and removing triggers

SCRIPT

```
show triggers
drop trigger no_overdraft
```

OUTPUT

```
+--------------+---------+------------------------------+--------------------------------+---------
    ----------------------------------------------------------------------------------------------
    -----+
| Trigger      | Table   | When it runs                 | Only if                        | Does

         |
+--------------+---------+------------------------------+--------------------------------+---------
    ----------------------------------------------------------------------------------------------
    -----+
| log_change   | account | after update                 | new.balance is not old.balance | insert i
    nto ledger (what, account_id, amount) values ("balance changed", new.id, new.balance - old.bala
    nce) |
| no_overdraft | account | after update                 | new.balance < 0                | raise "t
    he balance cannot go below zero"
         |
| tidy         | account | before insert                |                                | set new.
    owner = upper(new.owner)
         |
| restock      | stock   | after update, each statement |                                | insert i
    nto stock_log values ("update", count.rows)
```

```
        |
+--------------+---------+----------------------------+------------------------------+---------
    ----------------------------------------------------------------------------------------------
    -----+
4 row(s) in set

Query OK, trigger 'no_overdraft' dropped.
```

A trigger can change another table, and that change can set off the triggers of that table, up to 6 levels deep. A trigger that keeps setting itself off stops at that depth with an error, and the statement is taken back. Statements inside a trigger run silently. Triggers are saved in the database file, and a table that is dropped takes its triggers with it.

A procedure

`create procedure` saves statements under a name, with parameters. `:src` stands for the value that was written in the call. A line that ends with `as` or `;` goes on to the next line.

SCRIPT

```
create procedure transfer (src, dst, amount) as
  update account set balance = balance - :amount where id is :src;
  update account set balance = balance + :amount where id is :dst
call transfer(1, 2, 20)
find owner, balance from account
show procedures
```

OUTPUT

```
Query OK, procedure 'transfer' made. Run it with: call transfer(...)

Query OK, 1 row(s) updated.

Query OK, 1 row(s) updated.

+--------+---------+
| owner  | balance |
+--------+---------+
| Asha   | 50      |
| Ravi K | 70      |
| MEERA  | 0       |
| KIRAN  | 25      |
+--------+---------+
4 row(s) in set

+-----------+------------------+-------------------------------------------------------------------
    ---------------------------------------------------+
| Procedure | Parameters       | Does
                                                      |
+-----------+------------------+-------------------------------------------------------------------
    ---------------------------------------------------+
| transfer  | src, dst, amount | update account set balance = balance - :amount where id is :src;
     update account set balance = balance + :amount where  |
+-----------+------------------+-------------------------------------------------------------------
    ---------------------------------------------------+
1 row(s) in set
```

If a statement of the procedure fails, everything the call did is taken back. A call with the wrong number of values says how many it needs.

SCRIPT WITH A MISTAKE

```
call transfer(1)
```

WHAT enlngdb PRINTS

```
ERROR: procedure 'transfer' needs 3 value(s) (src, dst, amount) and got 1.

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

SCRIPT

```
drop procedure transfer
```

OUTPUT

```
Query OK, procedure 'transfer' dropped.
```

Did you mean

A table, a column or a command that is not there, but looks like one that is, gets a suggestion. Nothing is guessed: the statement still fails.

SCRIPT WITH A MISTAKE

```
find ownr from account
```

WHAT enlngdb PRINTS

```
ERROR: there is no column 'ownr'. Did you mean 'owner'? The columns are: id owner balance.

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

SCRIPT WITH A MISTAKE

```
fnd owner from account
```

WHAT enlngdb PRINTS

```
NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.

-- EnlngDB: statement not understood ---------------------------------
    1 | fnd owner from account
      | ^

What: Unrecognized or malformed database statement: 'fnd owner from account'
Why:  Statement does not match any valid conversational EnlngDB grammar.

Suggestions:
  * Did you mean 'find'? (the statement starts with 'fnd')
  * A statement begins with one of: use database, show, describe, create, alter, drop, insert into,
     find, count, with, update, change, delete, truncate, preview, undo, import, export, begin, com
    mit, rollback, explain, call
  * Comments start with '#' or '--' at the beginning of a line.
  * Every statement, with examples, is in the manual: enlngdb_manual.pdf (also at enlangg.site, Doc
    s).
------------------------------------------------------------------------
```

SCRIPT WITH A MISTAKE

```
find owner from acount
```

WHAT enlngdb PRINTS

```
ERROR: Table 'acount' does not exist in database 'book'.
       Did you mean 'account'?
       Tables in 'book': account, ledger, stock, stock_log

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

A word that differs by one letter (two letters for a word of five or more), or by two letters swapped, is suggested.

CHAPTER 17

The file on disk and the C interface

What is saved, and when

When a script ends without a failed statement, the engine writes the open database to `<name>.edb`. What is written is the change, to the log `<name>.edb-wal` beside the file, or the whole file now and then (see the end of this chapter's layout and chapter 15). Either one is written to a `.tmp` file first; when that is on the disk, it takes the place of the old one in one step. So a crash or a power cut while saving leaves the old state or the new one, whole, never half of one or none (the tests kill the program 30 times while it saves, and open the file after each; 60 more kills were run while saving through the log). A script that only reads (`find`, `select`, `show`, `count`, `describe`, `explain`, `export`, `preview`) does not write the file again. A script that has a failed statement saves nothing, and says so. A database dropped with `drop database ... confirmed` is not written again. Temporary tables, made inside a script for its own use, are not saved. Views, indexes, column types, rules and automatic-number counters are saved with the tables.

The layout of the file

Each number is written byte by byte, so the file reads the same on every machine and every compiler.

Part	Size	Content
magic	16 bytes	the text `ENLNG_C_EDB_V5` and two zero bytes
database name	64 bytes	text, padded with zero bytes
file id	16 bytes	random, new at each full write of the file (version 4 and on)
table count	4 bytes	a whole number
each table		the parts listed below

Each table has:

- its name (64 bytes) and the text of its view (4 bytes for the length, then the text; length 0 for a table).
- the number of columns (4 bytes) and for each column its name (64 bytes), five flag bytes (the type: 0 any, 1 integer, 2 real, 3 text, 4 boolean, 5 date; then primary key, unique, not null and automatic number), the default, the check and the referenced table and column (each a length of 4 bytes and then the text), and the next automatic number (8 bytes).
- the number of indexes (4 bytes) and for each its name, whether it is unique, and its columns.
- the number of rows (8 bytes), how many bytes the rows take (8), the width of a place (1 byte: 4 or 8, then 7 zero bytes), then every row: the number of cells (4 bytes), then for each cell a type byte (0 empty, 1 whole number, 2 decimal number, 3 text, 4 yes or no) and its value.

When the database has triggers or procedures, they follow the tables: the 16 bytes `ENLNGDB-EXT1` and zero bytes, the number of triggers (4 bytes) and for each its name and table (64 bytes each), its event (4 bytes: 1 insert, 2 update, 3 delete), its `when` condition and its statements (each a length of 4 bytes and then the text); then the number of procedures and for each its name, parameters and statements. Before triggers come last, after the 16 bytes `ENLNGDB-EXT2` and their number, in the same form, so that an older program that knows only after triggers never reads them. A file without them has nothing after the tables, and an older program that reads a file with them stops after the tables and ignores the rest. After the rows: the place of each row from the first (4 or 8 bytes each), so that a row can be read without reading the others; then the lookup indexes kept for the table (their count, 4 bytes; for each, its column, how many entries and how many rows without a hash, then the entries, each 8 bytes of hash and 4 of row, sorted by hash, then those rows). When the count has its top bit set, each index is followed by its column in order, for ranges and `like "abc%"`: the numbers (their count, then 8 bytes of value and 4 of row each, ascending), the rows with a text in the order of the texts, and the rows with another kind of value. A file of version 2 (`ENLNG_C_EDB_V2`) has only the number of rows before them (4 bytes, or 8 from a 64-bit program before) and nothing after; it is read whole, and written as version 5 at the next full write. A version 3 file is the same without the file id, and a version 4 file the same without the checksums below. Programs before this version do not read version 5; this one says of a file of a later version that it was written by a newer version, and leaves it as it is.

Version 5 adds checksums (CRC-32). After the indexes of each table come the number of blocks of 256 rows (4 bytes), a checksum for each block (of the bytes of its rows, then of their places) and one of the indexes. Just before the end mark, 4 bytes are the checksum of all the rest: the names, columns, triggers, procedures, users and rights. The rest is checked when the file is opened, and a file in which it changed is not opened. The rows are not read when the file opens: each block is checked the first time one of its rows is read, and a block that changed stops the program with "the file of this database is damaged", instead of a value being read as another. A changed index is not used, with a note, and the rows are read instead. In the tests, 1,200 random changes of bytes and 1,200 cuts gave no answer that differed from the good file without an error; with the same 400 changes, the version before refused 146 and gave other answers without an error for 64.

Beside a version 4 or 5 file there may be `<file>-wal`, the log of changes: the text `ENLNG_EDB_WAL_V1`, the id of the file it belongs to, and for each table the rows deleted, changed and added since the file was written, and a SHA-256 of the log at the end. The SHA-256 is checked first: a log whose SHA-256 does not fit makes the database not open (also when what changed is its id), and a whole log of another file is ignored.

A version 4 or 5 file ends with the 16 bytes `ENLNG_EDB_END_V4`. A file without them was cut, and is not opened. In the same way a file that is not a database at all, or whose tables cannot be read, is refused and left as it is: nothing runs on that database and nothing is written, so a damaged file is never replaced by an empty one. A row whose bytes do not make a row stops the program with an error. A database is named by the file it was opened from: a copy of `shop.edb` called `backup.edb` is the database `backup` and is saved to `backup.edb`. Triggers for each statement come after that, in a part that begins with `ENLNGDB-EXT4` (their events are 21, 22 and 23). An encrypted file begins with `ENLNG_C_EDB_ENC1`, then the salt (16 bytes), the PBKDF2 rounds (4), the nonce (12), and the whole file as above, sealed with ChaCha20-Poly1305, with its tag (16) at the end. Files written by earlier versions of EnlngDB begin with `ENLNG_C_EDB_V1`. They have no types, views or rules. They are read, and the next time the database is saved the file is written as version 2.

Using the engine from C

The engine is three files, `enlngdb.c`, `enlngdb_parser.c` and `enlngdb.h`, and the C standard library. This program makes a database, runs three statements and saves it. It prints what a script would print, because the last argument of `enlngdb_execute_statement` says whether to print.

```
#include "enlngdb.h"

int main(void) {
    EnlngDatabase* db = enlngdb_create("finance_db");
    enlngdb_execute_statement(db, "create table users with id, name, balance", true);
    enlngdb_execute_statement(db, "insert into users with id 1, name \"Asha\", balance 50000", true
    );
    enlngdb_execute_statement(db, "find name from users where balance is at least 40000", true);
    enlngdb_save(db, "finance_db.edb");
    enlngdb_free(db);
    return 0;
}
```

It is built together with the two engine files:

TERMINAL

```
gcc demo.c enlngdb.c enlngdb_parser.c -lm -o demo
```

and prints this when run (the times left out):

OUTPUT

```
Query OK, 0 rows affected. Table 'users' created with 3 columns.

Query OK, 1 row affected

+------+
| name |
+------+
| Asha |
+------+
1 row(s) in set
```

`enlngdb_execute_statement` returns true when the statement worked and false when it did not. `enlngdb_load(db, "finance_db.edb")` reads a file back into a database. The limits are in the table below.

Limits

Thing	The most
tables in a database	4,096
columns in a table	256
columns in the result of a query (a join)	512
indexes on a table	32
characters in a name	63

Thing	The most
characters in a statement, or in a text value	no limit but memory
characters of a `default`, a `check`, the query of a view	63, 95, 1023

A name, a default or a check that is too long is an error; nothing is cut short.

CHAPTER 18

Errors and refusals

EnlngDB does not guess. When a statement is unclear or would do something you may not have meant, it says what is wrong, and changes nothing. There are three kinds of answer.

- `ERROR:` the statement could not be done. The message says why, and usually what to write instead.
- `REFUSED:` the statement is clear, but it would remove data and the safety word `confirmed` is missing. Add the word to go ahead.
- a syntax error box, when the line is not any statement the engine knows. It lists the commands that exist.

A script that has a failed statement is still run to the end, so that you see every mistake at once, and then nothing is saved.

A word that is not a column

SCRIPT

```
create table student with id, name, marks
insert into student values (1, "Asha", 80), (2, "Ravi", 90)
```

OUTPUT

```
Query OK, 0 rows affected. Table 'student' created with 3 columns.

Query OK, 2 rows affected
```

SCRIPT WITH A MISTAKE

```
find nme from student
```

WHAT enlngdb PRINTS

```
ERROR: there is no column 'nme'. Did you mean 'name'? The columns are: id name marks.

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

The engine lists the columns the table really has. A mistyped column is never taken for a text.

A table that does not exist

SCRIPT WITH A MISTAKE

```
find name from students
```

WHAT enlngdb PRINTS

```
ERROR: Table 'students' does not exist in database 'book'.
       Did you mean 'student'?
       Tables in 'book': student

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

A statement that is not complete

SCRIPT WITH A MISTAKE

```
find name from
```

WHAT enlngdb PRINTS

```
ERROR: expected a table name after 'from'.

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

SCRIPT WITH A MISTAKE

```
update student set marks where id is 1
```

WHAT enlngdb PRINTS

```
ERROR: after 'marks' write '= <value>' or 'to <value>'.
       Nothing was changed.

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

A line that is not a statement

SCRIPT WITH A MISTAKE

```
tell me about student
```

WHAT enlngdb PRINTS

```
NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.

-- EnlngDB: statement not understood ---------------------------------
    1 | tell me about student
      | ^

What: Unrecognized or malformed database statement: 'tell me about student'
Why:  Statement does not match any valid conversational EnlngDB grammar.

Suggestions:
  * A statement begins with one of: use database, show, describe, create, alter, drop, insert into,
     find, count, with, update, change, delete, truncate, preview, undo, import, export, begin, com
    mit, rollback, explain, call
  * Comments start with '#' or '--' at the beginning of a line.
  * Every statement, with examples, is in the manual: enlngdb_manual.pdf (also at enlangg.site, Doc
    s).
-----------------------------------------------------------------------
```

Several rows, one of them wrong

A `values` list is all or nothing. The row with the mistake is named.

SCRIPT

```
create table price with item text, cost real not null
```

OUTPUT

```
Query OK, 0 rows affected. Table 'price' created with 2 columns.
```

SCRIPT WITH A MISTAKE

```
insert into price values ("pen", 2.5), ("book", null)
```

WHAT enlngdb PRINTS

```
ERROR: column 'cost' cannot be empty (it is marked not null).
       Nothing was inserted.

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

SCRIPT

```
count records in price
```

OUTPUT

```
+----------+
| count(*) |
+----------+
| 0        |
+----------+
1 row in set
```

An ambiguous name

When two tables both have a column of the same name, a bare name could mean either, so the engine asks you to say which.

SCRIPT

```
create table a with id, v
create table b with id, w
insert into a values (1, "x")
insert into b values (1, "y")
```

OUTPUT

```
Query OK, 0 rows affected. Table 'a' created with 2 columns.

Query OK, 0 rows affected. Table 'b' created with 2 columns.

Query OK, 1 row affected

Query OK, 1 row affected
```

SCRIPT WITH A MISTAKE

```
find id from a join b on a.id is b.id
```

WHAT enlngdb PRINTS

```
ERROR: more than one table has a column 'id'; write it as table.id.

NOTE: 1 statement(s) failed, so nothing from this run was saved to disk.
      Fix the errors above and run the script again.
```

SCRIPT

```
find a.id, v, w from a join b on a.id is b.id
```

OUTPUT

```
+------+---+---+
| a.id | v | w |
+------+---+---+
| 1    | x | y |
+------+---+---+
1 row(s) in set
```

What stays true

- A comparison with `null` is never true. Use `is null` and `is not null`.
- Dividing by zero is an error.
- A type that cannot be satisfied is an error. A text is never turned into a number without a reason.
- A key that exists replaces the row, and the engine tells you. It never makes a second row with the same key.
- Nothing is deleted for good without `confirmed`.

CHAPTER 19

From SQL to EnlngDB

If you know SQL, this chapter is the dictionary. The scripts after each table run most of its EnlngDB statements on the tables below, and none of them gave an error. The statements that remove everything are shown, and run, in chapter 11.

SCRIPT

```
create table users with id integer primary key, name text, balance integer, active boolean, dept te
    xt
insert into users values (1, "Asha", 500, true, "ops"), (2, "Ravi", 300, false, "dev"), (3, "Meera"
    , 900, true, "dev")
create table orders with id integer, user_id integer, total integer
insert into orders values (1, 1, 100), (2, 1, 250), (3, 3, 80)
```

OUTPUT

```
Query OK, 0 rows affected. Table 'users' created with 5 columns.

Query OK, 3 rows affected

Query OK, 0 rows affected. Table 'orders' created with 3 columns.

Query OK, 3 rows affected
```

Reading

SQL	EnlngDB
`SELECT * FROM users`	`find * from users`
`SELECT id, name FROM users WHERE id = 1`	`find id, name from users where id is 1`
`... WHERE balance >= 400 ORDER BY balance DESC LIMIT 2`	`find name from users where balance is at least 400 order by balance desc limit 2`
`SELECT DISTINCT dept FROM users`	`find distinct dept from users`
`SELECT COUNT(*) FROM users`	`count records in users`
`SELECT dept, COUNT(*) FROM users GROUP BY dept HAVING COUNT(*) > 1`	`find dept, count(*) from users group by dept having count(*) > 1`
`WHERE x BETWEEN 1 AND 5`, `IN (...)`, `LIKE 'A%'`, `IS NULL`	the same words: `between 1 and 5`, `in (...)`, `like "A%"`, `is null`

SQL	EnIngDB
`SELECT u.name, o.total FROM users u JOIN orders o ON u.id = o.user_id`	`find u.name, o.total from users u join orders o on u.id is o.user_id`
`... LEFT JOIN ...`	`... left join ...` or `users <- orders`
`UNION`, `INTERSECT`, `EXCEPT`	the same words
`WITH t AS (...) SELECT ...`	`with t as (...) find ...`
`CASE WHEN ... THEN ... END`	`case when ... then ... end`
`RANK() OVER (ORDER BY x)`	`rank() over (order by x)`

SCRIPT

```
find id, name from users where id is 1
find name from users where balance is at least 400 order by balance desc limit 2
count records in users
find dept, count(*) from users group by dept having count(*) > 1
find u.name, o.total from users u join orders o on u.id is o.user_id
find name, total from users <- orders
with t as (find name, balance from users where balance > 400) find name from t
```

OUTPUT

```
+----+------+
| id | name |
+----+------+
| 1  | Asha |
+----+------+
1 row(s) in set

+-------+
| name  |
+-------+
| Meera |
| Asha  |
+-------+
2 row(s) in set

+----------+
| count(*) |
+----------+
| 3        |
+----------+
1 row in set

+------+----------+
| dept | count(*) |
+------+----------+
```

```
| dev  | 2        |
+------+----------+
1 row(s) in set

+--------+---------+
| u.name | o.total |
+--------+---------+
| Asha   | 100     |
| Asha   | 250     |
| Meera  | 80      |
+--------+---------+
3 row(s) in set

+-------+-------+
| name  | total |
+-------+-------+
| Asha  | 100   |
| Asha  | 250   |
| Ravi  | null  |
| Meera | 80    |
+-------+-------+

4 row(s) in set

+-------+
| name  |
+-------+
| Asha  |
| Meera |
+-------+
2 row(s) in set
```

Changing

SQL	EnIngDB
`INSERT INTO users (id, name) VALUES (4, 'Dev')`	`insert into users with id 4, name "Dev"` or `insert into users (id, name) values (4, "Dev")`
`UPDATE users SET balance = 5000 WHERE id = 1`	`update users set balance = 5000 where id is 1`
`DELETE FROM users WHERE active = false`	`delete from users where active is false`
`DELETE FROM users`	`delete all from users confirmed`
`TRUNCATE TABLE users`	`truncate table users confirmed`
`ALTER TABLE users ADD COLUMN age INT`	`alter table users add column age integer`

SQL	EnIngDB
`DROP TABLE users`	`drop table users confirmed`

SCRIPT

```
insert into users with id 4, name "Dev"
insert into users (id, name) values (5, "Eli")
update users set balance = 5000 where id is 1
delete from users where active is false
alter table users add column age integer
find id, name, balance, age from users
```

OUTPUT

```
Query OK, 1 row affected

Query OK, 1 row affected

Query OK, 1 row(s) updated.

Query OK, 1 row(s) deleted.

Query OK, column 'age' added to table 'users'.

+----+-------+---------+------+
| id | name  | balance | age  |
+----+-------+---------+------+
| 1  | Asha  | 5000    | null |
| 3  | Meera | 900     | null |
| 4  | Dev   | null    | null |
| 5  | Eli   | null    | null |
+----+-------+---------+------+
4 row(s) in set
```

Things that differ

- **Text and conditions:** text is written in quotes, and conditions are written with words: `is`, `is not`, `is at least`. The marks `=`, `!=`, `>=` also work.
- `NULL`**:** `is null` and `is not null`, as in SQL. A comparison with `null` is never true, as in SQL.
- **Strings:** `+` joins two texts, and so does SQL's `||`.
- **A key:** a table's key is the column marked `primary key`, or a column called `id`. Inserting a row with a key that exists replaces that row, where SQL would refuse. The engine says that it replaced a row.
- **Statements that change a whole table:** `drop`, `truncate`, `drop column`, an `update` or `delete` with no `where` (or one every row meets), and an `import` that

writes over rows need the word `confirmed` at the end. SQL runs `UPDATE users SET active = 0` at once; EnlngDB asks for `update users set active = 0 confirmed`.

- **Types:** `integer`, `real`, `text`, `boolean`, `date`. A column with no type holds anything.
- **Users:** `create user`, `grant` and `revoke` work as in SQL, and the server checks them (chapter 14). A database can be larger than memory (chapter 15).
- **More than SQL:** `preview`, `undo`, CSV files and `confirmed` (chapter 15), and triggers, procedures and "did you mean" suggestions (chapter 16).
- **Quotes around names:** a table or column name with a space in it is not supported.

CHAPTER 20

What EnlngDB does not have

A list of what is missing is part of a reference. Each of these is something a statement or a program could be written for, and none of them exists. EnlngDB gives an error or a message when you try.

Not there at all

- **Certificates.** A client trusts a server the first time by its fingerprint, as ssh does, and checks its key after that (chapter 14).
- **Rights on a file.** Users and rights are checked by the server. Anyone who can read the `.edb` file can read the data, unless the file is encrypted (chapter 14); then anyone with the password can.
- **Zone names.** Time zones are offsets from UTC such as `+05:30`; names such as `Asia/Kolkata` are not known.
- **Arrays and user types.** A column holds one value of one of the eight types; a list goes into a `json` column.

There, but different from what you may expect

- **Indexes.** A `unique` index is a rule that is enforced. A plain `create index` is recorded and shown, and does not change anything else. The engine finds rows by `column is value`, `>`, `<`, `between`, `like` and `or` on its own where it pays (chapter 15); `not`, a `like` with fewer than three letters in a row, and a condition on a calculation read every row.
- **One program at a time, or the server.** A second program that opens a database in use waits up to 5 seconds and then stops with an error; many programs at once go through `enlngdb --serve`, which runs the requests that change something one at a time, and those that only read at the same time in reader processes (chapter 14).
- **A checksum of the rows.** A file that was cut, is not a database, or has a row that cannot be read is refused and never written over (chapter 17). A byte changed inside a value is not seen: only an encrypted file and the log of changes are checked as a whole. Keep copies of a file that matters.

- **Writing part of a file, always.** A script that changed rows writes only the change, to a log beside the file; the whole file is still written when the log passes a quarter of the file, when columns or tables change, and for an encrypted database. Rows are read only when needed, and lookups by a key, a unique column or a one-column index use the index kept in the file, and so do ranges and `like "abc%"` on those columns; `like "%abc%"`, and any condition on a column without a key, `unique` or an index, reads every row (chapter 15).
- **Counting and dates.** `datediff` counts days, `seconds_between` seconds; a moment with a zone is turned to UTC.

Limits

4,096 tables in a database, 256 columns in a table, 512 columns in a result, 32 indexes on a table, 63 characters in a name. A statement and a text value can be as long as memory allows (chapter 17). Triggers set each other off up to 6 levels deep.

Two engines, one language

EnlngDB also exists as a Python package for scripting. This book describes the C engine, `enlngdb.exe`, which is the one in the Enlangg toolchain, the playground and Studio on the Enlangg site. Features of the two engines are not the same, and a script that runs in one may not run in the other.

Where to go next

- Run the scripts of this book yourself: copy a script into a file, run `enlngdb file.enlngdb`.
- Open the playground on the Enlangg site, pick the preset "Rules, Subqueries & Views", and change it.
- Read the tests in the repository, `tests/test_enlngdb_sql.py` and `tests/test_enlngdb_queries.py`: each one is a small script with the answer it must give.